

SEQRITE

DRAGONFORCE RANSOMWARE

A Technical Analysis

WHITE PAPER

Analyst: **Ayush Singh Sachan**

Date: September, 2026 | Classification: TLP:CLEAR

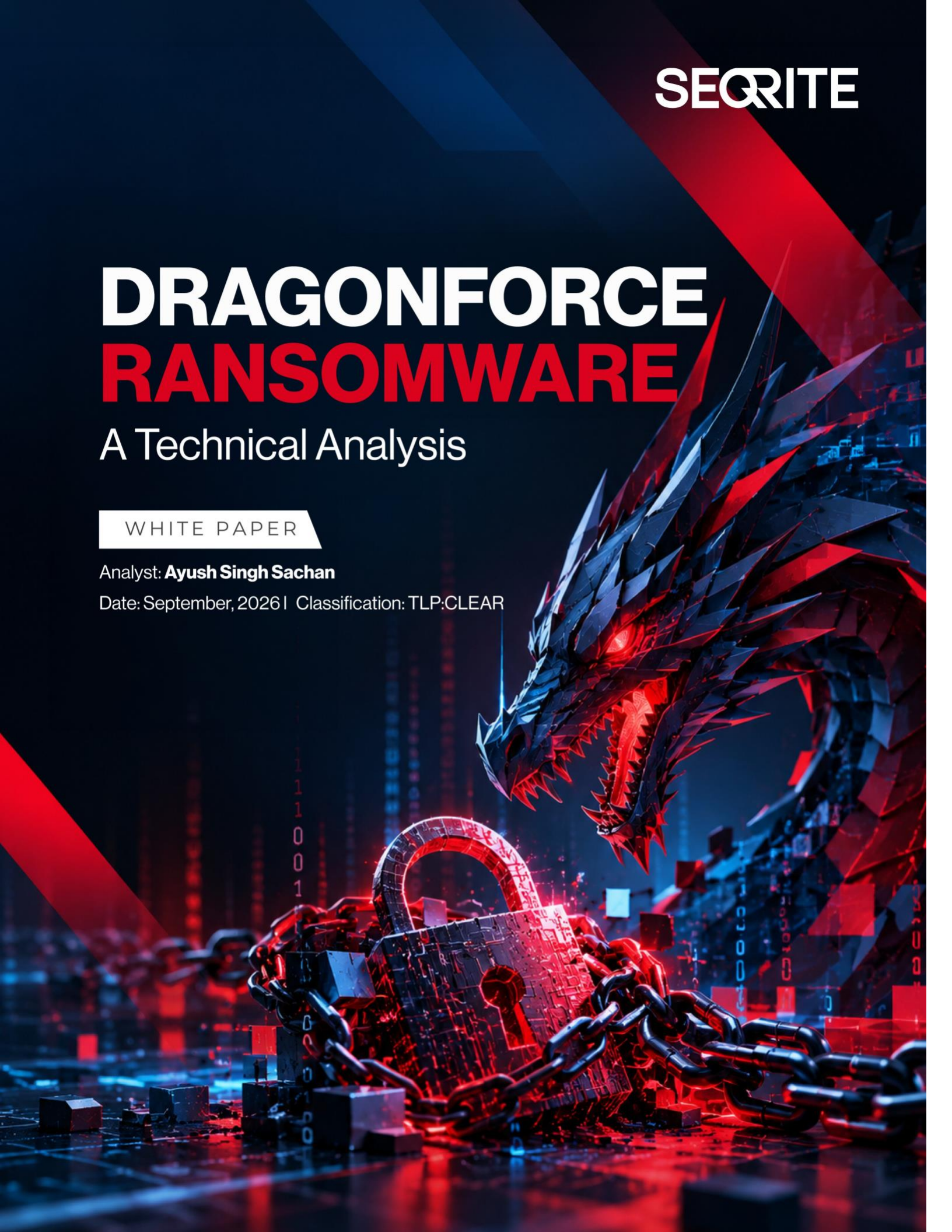


Table of Contents

Table of Contents	2
Executive Summary	5
Key Findings	6
Sample Overview	7
1. Introduction and Background	7
1.1 Background	7
1.2 Scope of This Document	7
1.3 Methodology.....	8
2. Execution Overview	8
3. Initialization and Anti-Static-Analysis Techniques	10
3.1 Process Startup	10
3.2 Dynamic API Resolution	12
3.3 String and Data Obfuscation Technique	14
3.4 Command-Line Argument Handling	16
4. Embedded Configuration.....	17
4.1 Building the ChaCha20 Decryption State.....	17
4.2 Decrypting and Parsing the Configuration	18
4.3 Configuration Contents.....	20
Priority-kill process list.....	21
Excluded paths, extensions, and filenames	21
4.4 Additional Strings from the Same Decrypted Blob	21
5. Encrypted Runtime Logging.....	22
5.1 Log File and Cryptographic State Setup.....	22
5.2 Constructing and Encrypting a Log Entry.....	23
5.3 Log Content and Forensic Value	23
6. Host and Environment Reconnaissance	24
6.1 User and Host Identity	24
6.2 Process Elevation Check	26
6.3 Executable Path Resolution and Architecture Check	26

6.4 Explorer Process Identification	27
7. Process Discovery and Termination.....	28
7.1 Setting Up the Monitoring Threads	28
7.2 Matching and Terminating Processes.....	29
7.3 Mutex Creation and Main-Thread Continuation	30
8. Anti-Recovery: Shadow Copy Deletion	31
8.1 COM/WMI Initialization and Locator Creation	31
8.2 Connecting, Querying, and Deleting Shadow Copies	31
9. Local and Network Target Discovery	32
9.1 Logical Drive Enumeration and Task Queue	32
9.2 Internal Network Discovery	33
9.3 SMB Share Enumeration and Propagation	34
10. Concurrent File Encryption	35
10.1 Encryption Worker Pool and Cryptographic Setup	35
10.2 Task Queue Architecture: Two Separate Queues.....	36
10.3 File Discovery and Filtering.....	38
10.4 The Ransom Note: Content, Branding, and a Traceable Identifier	39
10.5 File Encryption: Two Different Key Models	40
10.6 Renaming and Finalization.....	41
11. Shutdown and Cleanup.....	42
11.1 Terminating Idle Encryption Workers.....	42
11.2 Confirming the Two Worker Batches	43
11.3 Stopping the Process-Monitoring Threads	43
12. Desktop Customization	44
12.1 Extracting the Embedded Icon and Wallpaper	44
12.2 Icon Association: A Registry Key, Not a Per-File Icon	45
12.3 Wallpaper: Targets Enumerated User Contexts and the Active Session.....	45
12.4 Program Completion.....	47
13. Analytical Limitations and Unresolved Areas	47
13.1 Static Code Paths Never Dynamically Exercised.....	48

13.2 Purposes That Remain Unresolved	48
13.3 Imported Capabilities Not Dynamically Exercised	48
13.4 Scope Limitations of This Analysis	49
14. MITRE ATT&CK Mapping	50
15. Indicators of Compromise	51
File Paths and Names.....	51
Registry	51
Process and Synchronization Artifacts	52
Configuration Values	52
Network Behavior	52
16. Quick Heal Detections.....	53
Appendix A. Function and Address Reference	53
Appendix B. Selected Platform and API References	56

Executive Summary

The file analyzed in this document is a Windows ransomware sample belonging to DragonForce, a Ransomware-as-a-Service operation that emerged in 2023 and later adopted a "cartel" affiliate model. This sample's ransom note identifies the operation as "The DragonForce Ransomware Cartel." Unlike ransomware that only encrypts the local machine, this sample also targets other systems: it reads the local ARP cache, enumerates SMB shares on discovered hosts, and queues reachable remote shares for file processing. The observed execution demonstrated remote file processing through multiple SMB shares. No separate exploitation mechanism was observed in the sample.

From the moment it runs, the sample keeps meaningful strings - file paths, process names, and internal settings - encoded until use, and resolves eleven of its own Windows libraries at runtime using `LoadLibraryA` and `GetProcAddress()` rather than relying solely on static imports, exiting immediately if the very first resolution step fails (Section 3.2). A validation check confirms each resolved address points into the sample's own import table, consistent with ordinary use of the Import Address Table rather than a distinct anti-analysis technique. The sample then decrypts its configuration, records runtime activity in an encrypted log file, and interferes with processes in its configured priority list, including security, backup, database, and productivity software. It also queries Windows for existing Volume Shadow Copies and constructs a deletion command. Only after these preparations does it discover local and network targets and proceed into concurrent encryption, using sixteen worker threads split across two dedicated task queues.

The observed file-encryption path generates per-file key and state material at runtime and applies RSA processing using a public key embedded in the sample. The embedded key is a 0x214-byte `PUBLICKEYBLOB` carrying an RSA1 marker and fields consistent with a 4096-bit RSA modulus and public exponent 65537. This material is appended to the end of the target file by 0x00433E80 in two separate writes: a 0x20C-byte (524-byte) RSA-processed record, followed by a second, distinct 0x0D-byte (13-byte) trailing footer, before `SetEndOfFile()` finalizes the file. The corresponding RSA private key is not present in the sample. The file-renaming path uses the `.df_win` extension, for which the sample creates a custom file-type icon association, and the sample also attempts wallpaper-setting operations for the user and profile contexts it enumerates under `HKEY_USERS`, followed by `SystemParametersInfoW()` for the active session. The ransom note also claims data theft before encryption, a common double-extortion theme. No exfiltration mechanism was observed in the sample.

This document traces the sample's execution from process startup through its final action before returning from its main routine. Most of the behavior described here comes from direct observation of the sample's execution and artifacts, not from static inference alone.

Key Findings

- **Dynamic Library Loading with a Kill Switch:** Resolves LoadLibraryA manually, then loads eleven Windows libraries by decrypted name and calls GetProcAddress() for the functions it needs, rather than relying solely on static imports. A validation check confirms each resolved address points into the sample's own import table, so this does not hide capability from static analysis. If the first resolution step fails, the sample exits immediately without running any further code.
- **ChaCha20 Configuration and Log Encryption:** Encrypts its own configuration and every log entry with ChaCha20. The recovered configuration-decryption state was identical across the two observed executions of this sample. Log encryption uses a separate state.
- **Priority-Process Termination:** Maintains a configurable priority-process list (including security, backup, database, and productivity software) and attempts to terminate matching processes before file processing. A separate Restart Manager-related path is exercised when a target file cannot be opened because it is locked.
- **Volume Shadow Copy Deletion:** Queries Windows Volume Shadow Copies through WMI and constructs a WMIC.exe shadowcopy ... delete command from the returned shadow-copy ID.
- **ARP-Based Network Discovery:** Discovers other computers on the local network by reading the ARP cache rather than performing an active scan, then enumerates their SMB shares. The observed log records seventeen share discoveries across more than a dozen hosts.
- **Dual Encryption Queue Architecture:** Runs file encryption across sixteen concurrent worker threads split into two independent, dedicated task queues (one for local drives, one for network shares).
- **Hybrid RSA/ChaCha20 Per-File Encryption:** Generates per-file key and state material at runtime and applies RSA processing with the embedded public key, appended in two separate writes before the file is finalized with SetEndOfFile().
- **File Renaming and Ransom Note:** The observed encryption/renaming path generates randomized filenames with the .df_win extension and writes a readme.txt ransom note containing a build/instance key-derived identifier.
- **DragonForce Cartel Branding:** The ransom note identifies the operation as "The DragonForce Ransomware Cartel" and claims data was stolen prior to encryption. No exfiltration mechanism was observed in the sample.
- **Desktop Customization:** Extracts its own embedded icon and wallpaper images and registers the icon through HKEY_CLASSES_ROOT\.df_win\DefaultIcon, while attempting wallpaper changes for the user/profile contexts enumerated under HKEY_USERS.

Sample Overview

Property	Value
File type	32-bit Windows PE executable
Observed execution environment	Windows x64, running under WOW64
Command-line arguments used	None
Observed on-disk execution path	C:\Users\Administrator\Desktop\work\25280121706\sample.exe

1. Introduction and Background

1.1 Background

DragonForce is a Ransomware-as-a-Service (RaaS) operation, meaning its developers lease the malware and supporting infrastructure to affiliates who carry out the actual attacks. The group first surfaced in mid-to-late 2023, initially presenting itself as a hacktivist collective before shifting to a purely profit-driven model. Its earliest payloads were built on leaked LockBit 3.0 source code, later supplemented with code derived from the leaked Conti builder, and the group has since produced variants targeting Windows, Linux, ESXi, and NAS platforms.

DragonForce launched a formal affiliate program on an underground forum in mid-2024, and in March 2025 restructured again into a "ransomware cartel" model, offering white-label infrastructure that lets affiliates operate under their own brand while relying on DragonForce's backend tooling. Public reporting has also linked the group to living-off-the-land and Bring-Your-Own-Vulnerable-Driver (BYOVD) techniques, and to collaboration with other well-known criminal groups. By mid-2026, the group's leak site had claimed several hundred victims across dozens of countries, concentrated in professional services, manufacturing, and technology.

1.2 Scope of This Document

This document covers the sample's complete execution chain as reconstructed through dynamic and static analysis: startup and anti-analysis preparation, configuration and logging, reconnaissance, process termination, shadow-copy deletion, local and network target discovery, the concurrent file-encryption engine, and desktop customization. Analysis is complete through the end of DragonForce_Main().

1.3 Methodology

This document is based on primary analysis of the sample and its execution. The work combined live execution tracing, static inspection of the binary, recovery of the embedded configuration, decryption of runtime log entries, and examination of extracted blobs and embedded cryptographic material.

Because DragonForce repeats certain operations many times during execution, including file processing across sixteen worker threads, this document does not re-narrate every repetition. When a loop or recurring path was traced once and then shown to repeat, the narrative explains the mechanism once and uses the recovered runtime log to represent the aggregate result. Findings throughout this document are limited to behavior supported by the sample and its observed execution.

2. Execution Overview

Before the individual capabilities are examined in depth, it helps to see how they connect during a single run of the sample. The diagram below follows the main thread's path from process entry through to cleanup. It is a roadmap, not a strict timeline - several of these stages run concurrently once their worker threads are created.

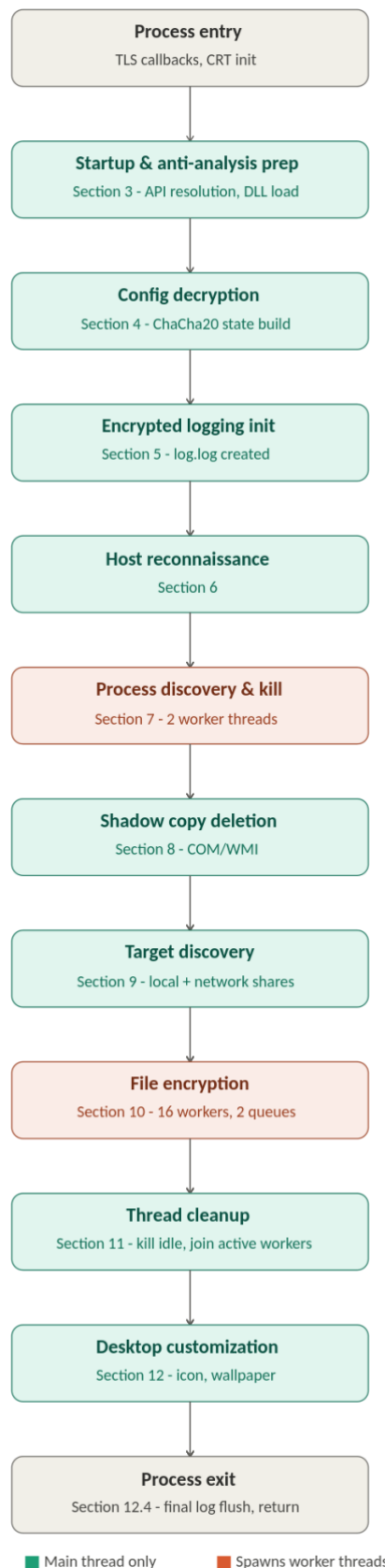


Figure 1. Execution flow of the sample from process entry through desktop customization and final process exit, annotated with the section covering each stage. Thread cleanup (Section 11) completes before desktop customization (Section 12) begins. Coral-highlighted stages spawn concurrent worker threads: process discovery/termination (2 threads) and file encryption (16 threads across two queues).

Reading the diagram. The main thread drives execution through local-drive and network-target production, while several background worker groups perform process monitoring, network discovery, and encryption concurrently. Two categories of background workers do the bulk of the destructive work: a small pair of threads repeatedly scans for and terminates configured priority processes (Section 7), while sixteen encryption workers are split across two dedicated queues (Section 10). A separate pair of network-discovery threads (Section 9.2 - 9.3) converts discovered addresses and enumerates shares while the encryption workers are already consuming published targets. After encryption, the main thread first winds down the background workers (Section 11) before moving into desktop customization (Section 12). The process exits only once that finishes.

Stage	Approx. Thread Involvement	Section
Startup, config, logging, recon	Main thread only	3 - 6
Process discovery/termination	2 worker threads	7
Shadow-copy handling via WMI	Main thread	8
Drive + network discovery	Main thread + 2 network-discovery threads	9
File encryption	16 worker threads (two 8-worker queues)	10
Worker shutdown/cleanup	Main thread	11
Desktop customization and finalization	Main thread	12

3. Initialization and Anti-Static-Analysis Techniques

This section covers everything DragonForce does before it starts acting on the target machine. Two techniques introduced here (dynamic API resolution and a repeating string-obfuscation routine) are used constantly throughout the rest of the sample, so they are explained in full once, here, and simply referenced by name in every later section.

3.1 Process Startup

When the sample is loaded, two Thread Local Storage (TLS) callbacks run first. Both perform ordinary compiler-generated runtime setup and contain nothing specific to DragonForce. Execution then reaches the PE entry point, which hands off to the C Runtime (CRT) startup routine at 0x0058E3D0.

The CRT startup retrieves the process command line and prepares an internal copy of it. This buffer is not used yet - it is parsed later, in Section 3.4, when DragonForce checks it for five supported switches (-p, -m, -log, -size, -nomutex). Once CRT setup finishes, control passes to DragonForce_Main() at 0x0045E160. Everything from this point on is the malware's own code.

Static inspection of the PE headers adds two relevant details here. First, the PE header's compile timestamp decodes to 21-01-1970 - a handful of days after the Unix epoch, not a real build date, and a standard technique for frustrating timeline attribution. Second, the LinkerVersion field (2.26) and the binary's reliance on the legacy msvcrt.dll for CRT functions, rather than vcruntime140.dll or ucrtbase.dll, both point to a MinGW or GCC-based toolchain rather than Visual Studio.

Function	Address	Role
TLS Callback 1 / 2	0x004D96E0 / 0x004D9730	Standard runtime setup, no DragonForce-specific behavior
PE Entry Point	0x004014B0	Hands off to CRT startup
CRT Startup	0x0058E3D0	Compiler-generated initialization that retrieves the command line
DragonForce_Main()	0x0045E160	Start of malware-specific execution

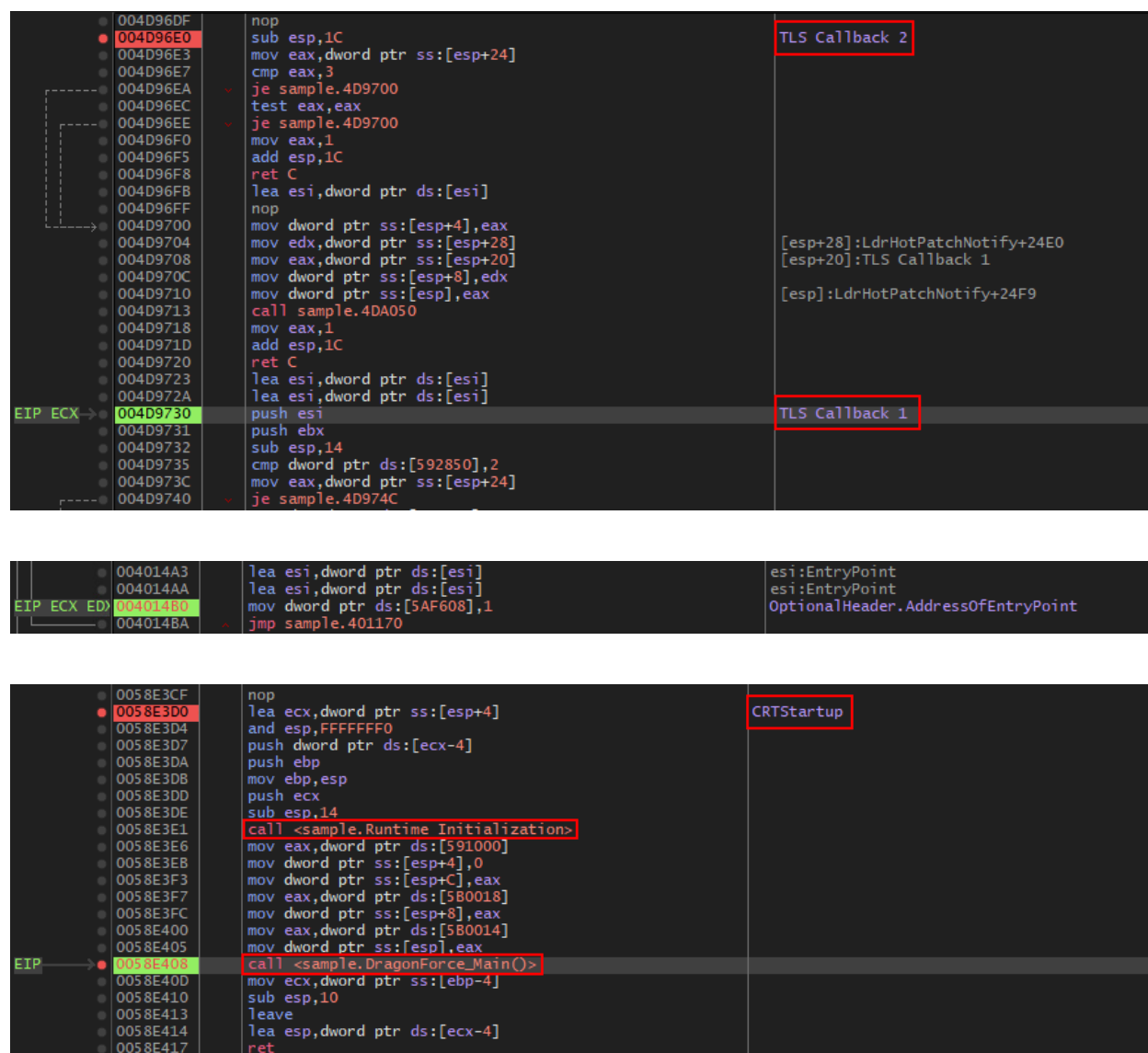


FIGURE 2. The two TLS callbacks and the PE entry point firing in order before `DragonForce_Main()` is reached.

3.2 Dynamic API Resolution

DragonForce loads several of its own Windows libraries at runtime using `LoadLibraryA` and `GetProcAddress()` rather than relying solely on static imports. As documented below, this does not hide any capability from static analysis and is best read as routine API resolution rather than a distinct anti-analysis technique.

The process happens in three stages. First, `InitializeApiResolver()` (0x00407D10) normalizes a handful of strings to lowercase, then resolves the address of `LoadLibraryA` from `kernel32.dll` using a minimal manual lookup rather than relying on a normal imported symbol. `DragonForce_Main()` checks this function's return value immediately after the call: if it is zero, meaning `LoadLibraryA` could not be

resolved, execution does not take the branch into the rest of DragonForce_Main(), and the process exits without performing any further action.

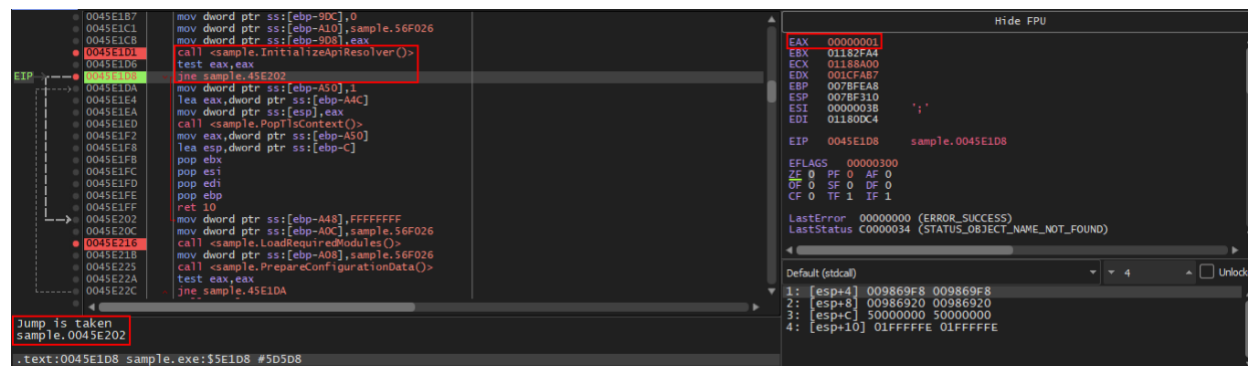


FIGURE 3. Execution Kill Switch in DragonForce_Main(): Disassembly at 0x0045E1D6 evaluating the return value of InitializeApiResolver(). If LoadLibraryA resolution fails (EAX = 0), the branch to 0x0045E202 is bypassed, cleanly terminating the process prior to executing malicious functionality.

Second, LoadRequiredModules() (0x00401AB0) uses that runtime-obtained LoadLibraryA to load eleven Windows libraries one at a time. Each DLL name is stored encrypted inside the binary and only decrypted immediately before the LoadLibraryA call that needs it.

DLL	Purpose
kernel32.dll	Core process, memory, file, and thread management
ws2_32.dll	Winsock networking
advapi32.dll	Registry, service, security, and CryptoAPI functions
rstrtmgr.dll	Restart Manager - recovers a file locked by another process (Section 10.3)
ole32.dll / oleaut32.dll	COM and OLE Automation, used for the WMI shadow-copy work (Section 8)
netapi32.dll	Network/share enumeration APIs
iphlpapi.dll	IP Helper - network interface and routing-table access
shlwapi.dll / shell32.dll	Shell and path utilities
w32time.dll	Windows Time service

Third, for each loaded DLL, DragonForce calls ResolveModuleExports() (0x004016E0), which calls GetProcAddress() to resolve the specific functions it needs. The routine then validates the returned address: it inspects the first byte for a 0xE9 (relative JMP) or 0xFF 0x25 (absolute indirect JMP) pattern, then compares the jump target against a reference value that falls inside 0x005B1000-0x005B3000, the

sample's own .idata (import data) section. This confirms that GetProcAddress() resolved to the expected import-table thunk. It is not a separate lookup mechanism, and every API resolved this way is already present in the sample's static import table.

```
InitializeApiResolver()
|
v
LoadRequiredModules() --(decrypt DLL name, LoadLibraryA)--> repeated x11
|
v
ResolveModuleExports() --(GetProcAddress, validate first byte == E9/FF,
                        compare target against .idata range 0x5B1000-0x5B3000)-->
repeated per DLL
```

The practical effect: DragonForce resolves these APIs through GetProcAddress() and validates the result against its own import-table thunks in .idata, rather than maintaining any separate lookup structure. Because every resolved API is already present in the sample's static import table, this mechanism adds no capability beyond ordinary use of the Import Address Table, though the binary is capable of creating and maintaining its own internal import table.

3.3 String and Data Obfuscation Technique

DragonForce keeps its meaningful strings - file paths, registry keys, API and DLL names, process names, command-line switches, mutex names, and more - encoded in the binary rather than stored as plain text. The same recovery routine is reused throughout the entire sample. It is documented in full once, here, and referred to simply as "the string-recovery routine" everywhere else in this document.

Each byte of an encoded string is processed through a reversible affine transform over modulo 127 (0x7F). The encoding path multiplies the character by 7, offsets it by 0x6A (106), and applies modular reduction using the compiler's reciprocal multiplication constant 0x81020409 (division by 127):

$$E(x) = (7x + 0x6A) \bmod 127$$

The reverse loop recovers the original byte by subtracting 0x6A and multiplying by 0xFFFFFEE (-18). Because $7 * (-18) = -126 = 1 \pmod{127}$, -18 (equivalent to 109 mod 127) functions as the exact modular multiplicative inverse:

$$D(y) = (109 * (y - 0x6A)) \bmod 127$$

```
; representative excerpt of the decode loop (NtQuerySystemInformation string, Section 7)
movzx ecx, byte ptr ss:[esp+ebx+E4]
sub    ecx, 6Ah
imul   ecx, ecx, FFFFFFFEEh    ; Multiply by modular inverse (-18)
mov    eax, ecx
imul   esi,          ; Reciprocal multiplication by 0x81020409 (div 127)
...
sub    ecx, eax
add    ecx, 7Fh              ; Normalize modulo 127
```

```
...
mov byte ptr ss:[esp+ebx+E4], cl
add ebx, 1
cmp ebx, 19
jne sample.47A2A0
```

The -p command-line switch provides a representative before-and-after example. At address 0x007BF2D9, the buffer appears as follows before and after the decode loop:

```
Encoded 0x007BF2D9: 28 6A 01 6A 6A 2C 2C 18 00 77 10 0F 1D 00 00
Decoded 0x007BF2D9: 2D 00 70 00 00 00 2C 2C 18 00 77 10 0F 1D 00 00
                  (decodes to the UTF-16 string "-p", followed by
                  the start of the next encoded switch in the buffer)
```

007BF2C9	21 5B 1A 54	28 09 27 20	3F 4D 4D 0F	0E 78 5F 00	! [.T(' ?MM..x_.
007BF2D9	28 6A 01 6A	6A 6A 2C 2C	18 00 77 10	0F 1D 00 00	(j.jj],...w....
007BF2E9	00 00 00 4C	03 00 00 03	5D 5A 77 29	F0 8E 76 79	...L....]Zw)δ.vy
007BF2C9	21 5B 1A 54	28 09 27 20	3F 4D 4D 0F	0E 78 5F 00	! [.T(' ?MM..x_.
007BF2D9	2D 00 70 00	00 00 2C 2C	18 00 77 10	0F 1D 00 00	-p...w....
007BF2E9	00 00 00 4C	03 00 00 03	5D 5A 77 29	F0 8E 76 79	...L....]Zw)δ.vy

FIGURE 4. Memory view at 0x007BF2D8 captured immediately before and immediately after the decode loop, showing the same 16 bytes changing from the encoded form to the decoded form.

This routine runs in one of two modes, depending on where the string already sits at the moment it is needed. When the string is already resident as plaintext, which includes configuration values, success and error log messages, and other strings kept as plain text in the .data, .rdata, and .bss sections, the routine encrypts it byte by byte into a newly allocated buffer, then immediately decrypts each moved byte back to plaintext at that new location for the caller to use. This encrypt-then-decrypt-at-the-new-location cycle recurs throughout execution wherever such a string is used. When the string is instead embedded directly in the code as already-encrypted bytes, as seen with the mutex name, printf-style format strings such as %%02d:%%02d:%%02d, and API names referenced directly in code such as NtQueryInformationFile, only the decryption half of the routine runs: the encrypted bytes are moved to a newly allocated buffer and decrypted there, with no encryption step beforehand. The plaintext strings in the first category are not stored as plaintext from the start. A separate function decrypts them once, from their encrypted form embedded in the binary, and writes the result into the .bss section. From that point on, every subsequent use of the string goes through the encrypt-then-decrypt-at-a-new-location cycle described above.

Where it recurs	Address	Recovers
Command-line switch comparison	0x00458B30 area	-p, -m, -log, -size, -nomutex
DLL name decryption	inside LoadRequiredModules()	The eleven DLL names in Section 3.2

Where it recurs	Address	Recovers
CSP provider name	0x00485BE0	"Microsoft Enhanced RSA and AES Cryptographic Provider" (Section 10)
Native API/module name	0x0047A2A0 area	NtQuerySystemInformation, ntdll.dll (Section 7)
Mutex name	main-thread execution, Section 7	hsfjuukjzloqu28oajh727190

3.4 Command-Line Argument Handling

After decoding each supported switch (Section 3.3), DragonForce passes it to `CompareCommandLineArgument()` (0x0048940, though the address is shorter than the eight-digit format used elsewhere and is retained as observed), which first checks that at least one argument was actually supplied, then performs a case-insensitive comparison using `lstrcmpiW()`. This is repeated for all five supported switches.

The sample was executed with no command-line arguments, so `CompareCommandLineArgument()` exited immediately on its argument-count check for every switch, and execution continued along the default path documented throughout the rest of this document.

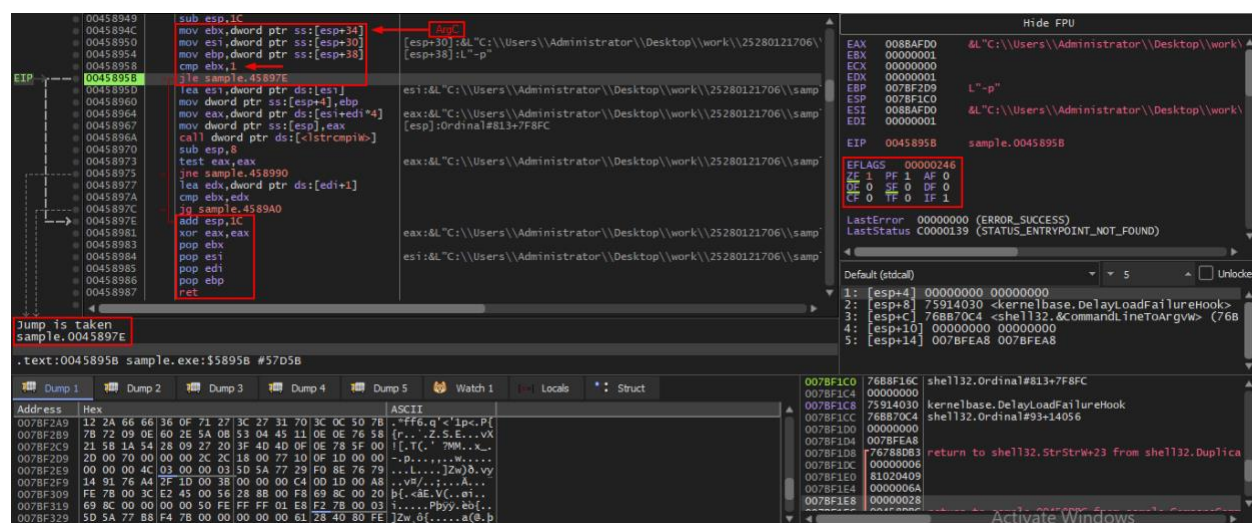


FIGURE 5. Debugger view showing the decoded switch strings and the argument-count check taking the no-argument branch.

These five switches - and the priority-kill list format and WMIC shadow-copy deletion syntax noted elsewhere in this document - match the command-line interface of the leaked Conti ransomware builder identified as a code source for this family (Section 1.1). Published reverse engineering of that codebase documents the function of each switch as follows:

Switch	Argument	Function
-p	directory	Encrypts only the specified directory, using a single thread.
-m	local	Encrypts all available local drives using multiple threads.
-m	net	Encrypts reachable network shares using multiple threads.
-m	all	Combines local and network encryption. This is the default mode when no switch is supplied.
-m	backups	Present in the parsing logic, though multiple independent analyses of the Conti codebase report no corresponding encryption behavior implemented behind it.
-size	chunk value	Switches large files to chunk-based (partial) encryption instead of encrypting the full file.
-log	logfile path	Writes timestamped activity entries to the specified log file.
-nomutex	(none)	Skips creation of the synchronization mutex, allowing more than one instance of the ransomware to run concurrently on the same host.

Sourcing note: this table reflects published third-party analysis of the shared Conti codebase, not this sample's own dynamic trace - the analyzed run was launched without arguments, so DragonForce's specific implementation of each switch was not independently exercised here.

Sources: CyberGeeks Technical Blog, "Dissecting the last version of Conti Ransomware using a step-by-step approach" (2021). Zscaler ThreatLabz, "Despite Leaks, Conti Ransomware Attacks Persist" (2022). Splunk Threat Research, Conti command-line detection content.

4. Embedded Configuration

Before DragonForce does anything else, it needs its settings - which processes to prioritize killing, which file extension to use, how aggressively to encrypt, and dozens of other values. None of these are stored as plain text in the binary. They are encrypted as a single block and only decrypted in memory once the required DLLs are loaded (Section 3).

4.1 Building the ChaCha20 Decryption State

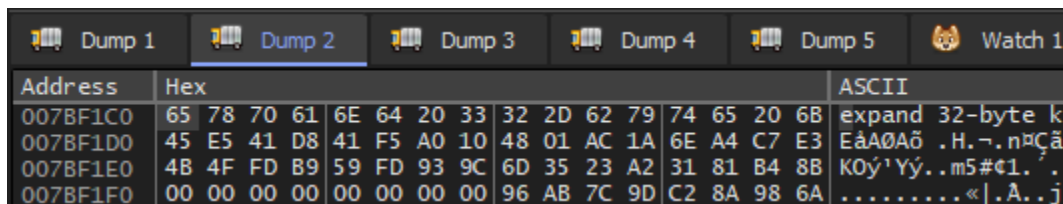
DragonForce uses the ChaCha20 stream cipher to protect its embedded configuration, and it reuses the exact same cipher (with a different key/state) to encrypt every entry in its runtime log later (Section 5).

PrepareConfigurationData() builds the ChaCha20 state in two steps. InitializeChaChaState() copies the standard ChaCha20 constant string "expand 32-byte k" together with the embedded 32-byte key into a

state buffer. AppendChaChaState() then fills in the remaining state words required to complete the 64-byte ChaCha20 state.

```
007BF1C0 65 78 70 61 6E 64 20 33 32 2D 62 79 74 65 20 6B expand 32-byte k
007BF1D0 45 E5 41 D8 41 F5 A0 10 48 01 AC 1A 6E A4 C7 E3 EâA0Að .H.~.nðÇã
007BF1E0 4B 4F FD B9 59 FD 93 9C 6D 35 23 A2 31 81 B4 8B KOý¹Yý..m5#ç1.´.
007BF1F0 00 00 00 00 00 00 00 00 96 AB 7C 9D C2 8A 98 6A .....«|.Ã..j
```

The same 64-byte configuration-decryption state was observed across two executions of this sample.



Address	Hex	ASCII
007BF1C0	65 78 70 61 6E 64 20 33 32 2D 62 79 74 65 20 6B	expand 32-byte k
007BF1D0	45 E5 41 D8 41 F5 A0 10 48 01 AC 1A 6E A4 C7 E3	EâA0Að .H.~.nðÇã
007BF1E0	4B 4F FD B9 59 FD 93 9C 6D 35 23 A2 31 81 B4 8B	KOý¹Yý..m5#ç1.´.
007BF1F0	00 00 00 00 00 00 00 00 96 AB 7C 9D C2 8A 98 6A	« .Ã..j

FIGURE 6. Memory dump of the completed 64-byte ChaCha20 state buffer, with the "expand 32-byte k" constant and embedded key/state words visible.

4.2 Decrypting and Parsing the Configuration

DecryptEmbeddedConfiguration() (0x00487FE0) takes the encrypted configuration blob, the ChaCha20 state above, and the blob's length, and runs the standard ChaCha20 quarter-round to produce plaintext into a newly allocated heap buffer. The decrypted buffer is one long string with every configuration entry separated by a pipe character. SplitConfigurationString() walks that buffer, splitting on the delimiter and copying each entry into its own heap allocation.

 Dump 1

Address	Hex	ASCII
00A60F10	AD DE 66 EA 57 C2 CE A5 3A 9E 00 68 1A 00 02 1A	.df&AI&:..h....
00A60F20	00 53 6F 66 74 77 61 72 65 5C 4D 69 63 72 6F 73	.Software\Micros
00A60F30	6F 66 74 5C 4D 61 6E 61 67 65 72 08 00 73 65 71	oft\Manager..seq
00A60F40	75 65 6E 63 65 04 00 6E 65 78 74 0A 00 00 00 01	uence..next.....
00A60F50	17 00 43 3A 5C 55 73 65 72 73 5C 50 75 62 6C 69	..C:\Users\Publi
00A60F60	63 5C 6C 6F 67 2E 6C 6F 67 01 01 00 00 20 00 00	c\log.log.... ..
00A60F70	00 A0 00 00 00 30 00 14 01 01 3E 42 00 00 3E 42	0....>B...>B
00A60F80	00 00 18 00 43 3A 5C 55 73 65 72 73 5C 50 75 62	C:\Users\Pub
00A60F90	6C 69 63 5C 69 63 6F 6E 2E 69 63 6F 00 01 00 DF	lic\icon.ico...B
00A60FA0	01 4D 73 4D 70 45 6E 67 2E 65 78 65 7C 73 71 6C	.MsMpEng.exe sql
00A60FB0	2E 65 78 65 7C 6F 72 61 63 6C 65 2E 65 78 65 7C	.exe oracle.exe
00A60FC0	6F 63 73 73 64 2E 65 78 65 7C 64 62 73 6E 6D 70	ocssd.exe dbsnmp
00A60FD0	2E 65 78 65 7C 73 79 6E 63 74 69 6D 65 2E 65 78	.exe synctime.ex
00A60FE0	65 7C 61 67 6E 74 73 76 63 2E 65 78 65 7C 69 73	e agntsvc.exe is
00A60FF0	71 6C 70 6C 75 73 73 76 63 2E 65 78 65 7C 78 66	qlplussvc.exe xf
00A61000	73 73 76 63 63 6F 6E 2E 65 78 65 7C 6D 79 64 65	ssvcon.exe myde
00A61010	73 68 74 6F 70 73 65 72 76 69 63 65 2E 65 78 65	sktopservice.exe
00A61020	7C 6F 63 61 75 74 6F 75 70 64 73 2E 65 78 65 7C	locatoupds.exe
00A61030	65 6E 63 73 76 63 2E 65 78 65 7C 66 69 72 65 66	encsvc.exe firef
00A61040	6F 78 2E 65 78 65 7C 74 62 69 72 64 63 6F 6E 66	ox.exe tbirdconf
00A61050	69 67 2E 65 78 65 7C 6D 79 64 65 73 68 74 6F 70	ig.exe mydesktop
00A61060	71 6F 73 2E 65 78 65 7C 6F 63 6F 6D 6D 2E 65 78	qos.exe ocomm.ex
00A61070	65 7C 64 62 65 6E 67 35 30 2E 65 78 65 7C 73 71	e dbeng50.exe sq
00A61080	62 63 6F 72 65 73 65 72 76 69 63 65 2E 65 78 65	bcoreservice.exe
00A61090	7C 65 78 63 65 6C 2E 65 78 65 7C 69 6E 66 6F 70	excel.exe infop
00A610A0	61 74 68 2E 65 78 65 7C 6D 73 61 63 63 65 73 73	ath.exe msaccess
00A610B0	2E 65 78 65 7C 6D 73 70 75 62 2E 65 78 65 7C 6F	.exe mspub.exe o
00A610C0	6E 65 6E 6F 74 65 2E 65 78 65 7C 6F 75 74 6C 6F	nenote.exe outlo
00A610D0	6F 68 2E 65 78 65 7C 70 6F 77 65 72 70 6E 74 2E	ok.exe powerpnt.
00A610E0	65 78 65 7C 73 74 65 61 6D 2E 65 78 65 7C 74 68	exe steam.exe th
00A610F0	65 62 61 74 2E 65 78 65 7C 74 68 75 6E 64 65 72	ebat.exe thunder
00A61100	62 69 72 64 2E 65 78 65 7C 76 69 73 69 6F 2E 65	bird.exe visio.e
00A61110	78 65 7C 77 69 6E 77 6F 72 64 2E 65 78 65 7C 77	xe winword.exe w
00A61120	6F 72 64 70 61 64 2E 65 78 65 7C 6E 6F 74 65 70	ordpad.exe notep
00A61130	61 64 2E 65 78 65 7C 63 61 6C 63 2E 65 78 65 7C	ad.exe calc.exe
00A61140	77 75 61 75 63 6C 74 2E 65 78 65 7C 6F 6E 65 64	wuauct.exe oned
00A61150	72 69 76 65 2E 65 78 65 7C 53 51 4C 41 47 45 4E	rive.exe SQLAGEN
00A61160	54 2E 65 78 65 7C 73 71 6C 73 65 72 76 72 2E 65	T.exe sqlservr.e
00A61170	78 65 7C 53 51 4C 57 72 69 74 65 72 2E 65 78 65	xe SQLWriter.exe
00A61180	00 00 07 00 2E 64 66 5F 77 69 6E 01 65 00 74 6D	df_win.e.tm
00A61190	70 7C 77 69 6E 6E 74 7C 74 65 6D 70 7C 74 68 75	p winnt temp thu
00A611A0	6D 62 7C 24 52 65 63 79 63 6C 65 2E 42 69 6E 7C	mb \$Recycle.Bin
00A611B0	24 52 45 43 59 43 4C 45 2E 42 49 4E 7C 53 79 73	\$RECYCLE.BIN Sys
00A611C0	74 65 6D 20 56 6F 6C 75 6D 65 20 49 6E 66 6F 72	tem Volume Infor
00A611D0	6D 61 74 69 6F 6E 7C 42 6F 6F 74 7C 57 69 6E 64	mation Boot Wind
00A611E0	6F 77 73 7C 70 65 72 66 6C 6F 67 73 7C 50 75 62	ows perflogs Pub
00A611F0	6C 69 63 25 00 2E 65 78 65 7C 2E 64 6C 6C 7C 2E	lic%..exe .dll .
00A61200	6C 6E 6B 7C 2E 73 79 73 7C 2E 6D 73 69 7C 2E 62	lnk .sys .msi .b
00A61210	61 74 7C 2E 64 66 5F 77 69 6E 0A 00 72 65 61 64	at .df_win..read
00A61220	6D 65 2E 74 78 74 00 00 01 A4 45 00 00 A4 45 00	me.txt...&E..&E.
00A61230	00 23 00 43 3A 5C 55 73 65 72 73 5C 50 75 62 6C	..#C:\Users\Publ
00A61240	69 63 5C 77 61 6C 6C 70 61 70 65 72 5F 77 68 69	ic\wallpaper_whi

FIGURE 7. Memory view of the heap buffer immediately after `DecryptEmbeddedConfiguration()` returns, showing the plaintext delimiter-separated configuration string.

This same decryption routine (`ChaCha20Crypt()` at `0x00487990`) is also the one reused for log encryption in Section 5 - the cryptographic core is identical, which is itself useful evidence when identifying the cipher.

4.3 Configuration Contents

The table below is the complete decrypted configuration observed for this sample.

Setting	Observed Value	Meaning
build_key	9E3AA5CEC257EA66	Identical across the two analyzed executions (Section 4.1)
instance_key	561C24DEEECC136E	Observed configuration value for the analyzed sample
custom_icon	1	Enables custom icon behavior (Section 12)
custom_wallpaper	1	Enables custom wallpaper behavior (Section 12)
custom_extension	.df_win	Extension used by the file-renaming path (Section 10)
time_sync	0	Disabled in this build
encrypt_mode	10	Selects the encryption strategy used in Section 10
full_encrypt_threshold	2097152 bytes (2 MB)	The analyzed run demonstrated the full-encryption branch (10.5)
header_encrypt_threshold	10485760 bytes (10 MB)	Threshold associated with the large-file/header-encryption path
header_encrypt_size	3145728 bytes (3 MB)	Amount associated with header encryption
other_encrypt_chunk_percent	20%	Percentage associated with the alternate/partial encryption path
encrypt_file_names	1	Filenames are renamed as part of encryption (Section 10)
schedule_job	0	Scheduled-task persistence disabled in this build
kill	1	Enables priority-process termination (Section 7)
use_sys	0	Disabled in this build
whitelist	1	Enables the excluded-paths list below

Setting	Observed Value	Meaning
file.name (ransom note)	readme.txt	Ransom note filename dropped during encryption (Section 10)

Priority-kill process list

Used in Section 7. Includes 38 entries covering security, backup, database, and office-productivity software:

Priority Processes (1)	(2)	(3)	(4)
MsMpEng.exe	sql.exe	oracle.exe	ocssd.exe
dbnmp.exe	synctime.exe	agntsvc.exe	isqlplussvc.exe
xfssvccon.exe	mydesktopservice.exe	ocautoupds.exe	encsvc.exe
firefox.exe	tbirdconfig.exe	mydesktopqos.exe	ocomm.exe
dbeng50.exe	sqbcoreservice.exe	excel.exe	infopath.exe
msaccess.exe	mspub.exe	onenote.exe	outlook.exe
powerpnt.exe	steam.exe	thebat.exe	thunderbird.exe
visio.exe	winword.exe	wordpad.exe	notepad.exe
calc.exe	wuauclt.exe	onedrive.exe	SQLAGENT.exe
sqlservr.exe	SQLWriter.exe		

Excluded paths, extensions, and filenames

Used in Section 10 to skip system-critical locations and avoid double-encrypting or breaking its own files:

Excluded Paths	Excluded Extensions	Excluded Filenames
tmp, winnt, temp, thumb, \$Recycle.Bin, \$RECYCLE.BIN, System Volume Information, Boot, Windows, perflogs, Public	.exe, .dll, .lnk, .sys, .msi, .bat, .df_win	readme.txt

4.4 Additional Strings from the Same Decrypted Blob

Several additional standalone values appear in the same decrypted buffer even though they are not part of the formatted configuration block:

Value	Likely Role
Software\Microsoft\Manager	Registry key path stored in the decrypted buffer.
sequence	Standalone string stored in the decrypted buffer.
next	Standalone string stored in the decrypted buffer.
C:\Users\Public\wallpaper_white.png	Destination path used by the custom WallpaperRegistration() (Section 12).
C:\Users\Public\icon.ico	Destination path used by the custom icon routine (Section 12).

5. Encrypted Runtime Logging

From this point forward, DragonForce records major execution stages in an encrypted runtime log stored on disk. The log captures reconnaissance, process handling, target discovery, and file-processing activity, making it one of the clearest execution records produced by the sample itself.

5.1 Log File and Cryptographic State Setup

InitializeLogFile() decrypts the embedded path string C:\Users\Public\log.log and copies it into a heap buffer. InitializeLogFile() then opens that path with CreateFileW() for read/write access.

PrepareLogEncryptionState() writes two 8-byte values to the start of the file in plaintext with WriteFile(): the build_key and instance_key values from Section 4.3. Only after that 16-byte header is written does the malware prepare the log's ChaCha20 state, using the same two-step mechanism from Section 4.1 but in a separate 64-byte heap buffer dedicated to logging.

```
WriteFile(build_key, 8 bytes) --> WriteFile(instance_key, 8 bytes) --> plaintext
header in log.log
|
v
InitializeChaChaState() + AppendChaChaState() --> log-specific ChaCha20 state
|
v
Ready for encrypted log-entry writes
```

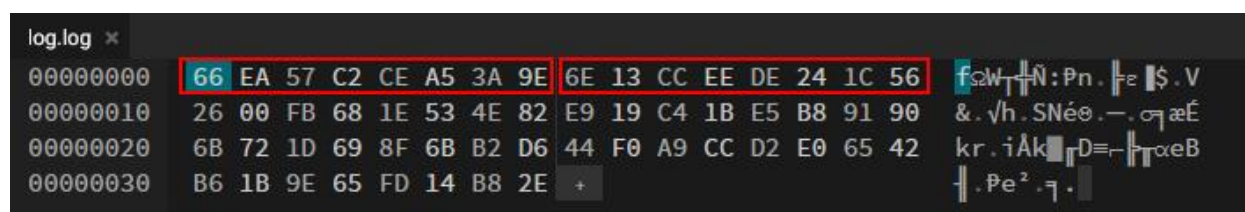


FIGURE 8. Hex view of `C:\Users\Public\log.log` showing the first 16 plaintext bytes on disk, highlighting the `build_key` and `instance_key` header before the encrypted entries begin.

5.2 Constructing and Encrypting a Log Entry

Every log write follows the same three-function path and is gated by a one-byte enable flag:

Wrapper	Address	Flag Checked
<code>WriteEncryptedLogEntry()</code>	<code>0x00495440</code>	byte ptr [<code>0x5AF581</code>]
<code>LogFormattedMessage()</code>	<code>0x004A56A0</code>	byte ptr [<code>0x5AF582</code>]

Both wrappers call `PrepareEncryptedLogEntry()` (`0x00494270`), which gathers the current local time and thread identifier, formats the prefix string `"[%02d:%02d:%02d] [Th:%04X]"`, and converts the final Unicode string to a multibyte buffer with `wcstombs()`. The plaintext buffer is then passed to `EncryptLogBuffer()` (`0x00487FB0`), which forwards it into `ChaCha20Crypt()` together with the log-specific state from Section 5.1. The resulting ciphertext is written to `log.log` with `WriteFile()`.

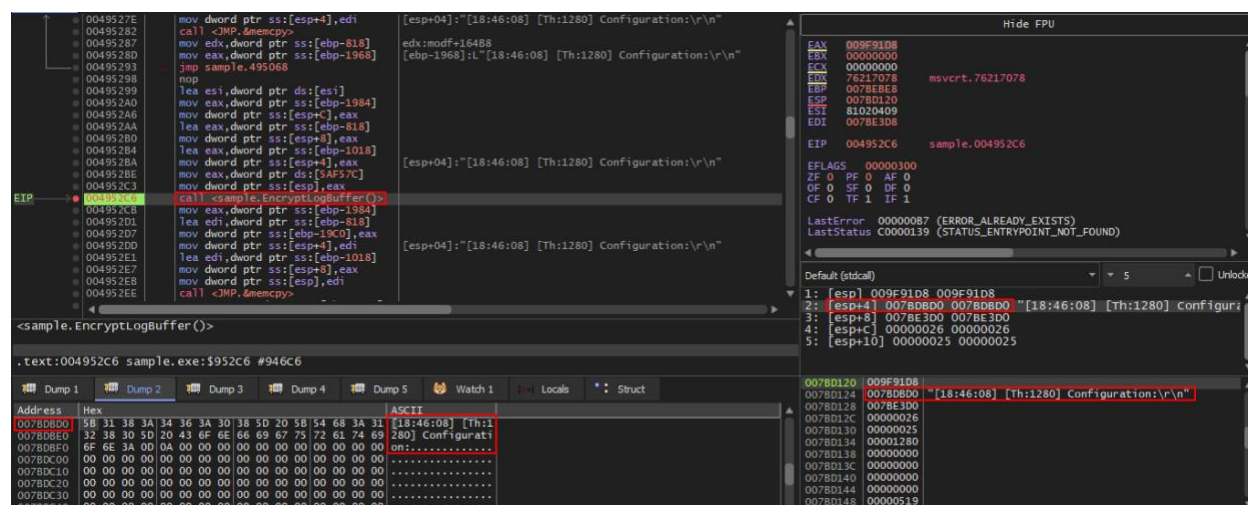


FIGURE 9. Debugger view showing `PrepareEncryptedLogEntry()` building the timestamp/thread prefix, followed by the call into `EncryptLogBuffer()` before the encrypted buffer is written to `log.log`.

5.3 Log Content and Forensic Value

The entries below were captured via debugger breakpoints immediately before each was encrypted and written to `log.log`. They reflect the same timestamped execution record the sample writes.

Representative entries include:

```
[+] GetTokenInformation returned required size of: 36
Running under: XDRAPRDWV010T
process is elevated: 1
```

```
Current executable path: C:\Users\Administrator\Desktop\work\25280121706\sample.exe
Starting proc: (Creates Threads)
Starting kill proc: 1
=====
(1st Thread)
Starting priority proc
[+] Process 'MsMpEng.exe' PID: 3120
[-] OpenProcess failed: MsMpEng.exe
[+] Process 'Firefox.exe' PID: 9128
[+] Terminating PID: 9128 for process 'Firefox.exe'
[+] Process Terminated: Firefox.exe
=====
Found 2 drives:
C:\
D:\
```

For defenders, C:\Users\Public\log.log is a high-value artifact: when the corresponding key material is available, the file can be decrypted offline to reconstruct the sample's execution chronology on the compromised host.

6. Host and Environment Reconnaissance

Before terminating processes or encrypting files, DragonForce collects several host and execution-context values: account information, token elevation state, executable path, and processor architecture.

6.1 User and Host Identity

CollectUserAndHostInformation() (0x0048CAA0) takes the current process ID, opens the process, and retrieves its access token. It calls GetTokenInformation() twice - once to learn the required buffer size, and once to actually retrieve the token's user SID.

Call	Observed Value
GetTokenInformation() required size	36 bytes
Observed host-related string	XDRAPRDWV010T
LookupAccountSidW() -> username	ADMINISTRATOR

The resolved SID is passed to LookupAccountSidW() to resolve account and domain information. DragonForce logs XDRAPRDWV010T as "Running under: XDRAPRDWV010T."

Address	Hex	ASCII
007BEE00	41 00 64 00	A.d.m.i.n.i.s.t.
007BEE04	72 00 61 00	r.a.t.o.r....{
007BEE08	0C 17 0C 59	...Y.H.....yy

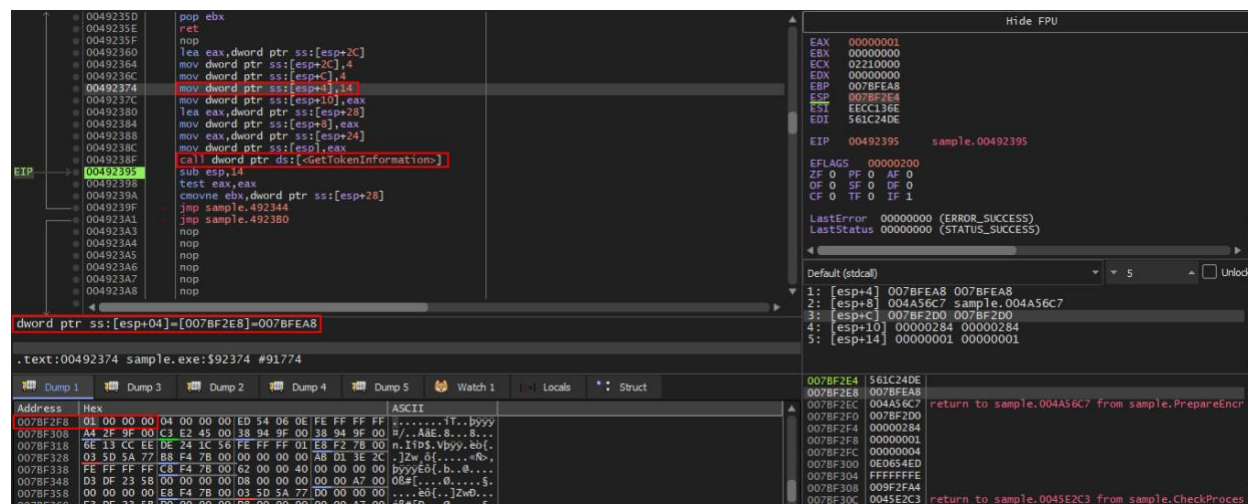
Address	Hex	ASCII
007BF0E8	58 00 44 00	X.D.R.A.P.R.D.W.
007BF0EC	56 00 30 00	V.O.1.0.T....f.o

FIGURE 10. Call to `GetTokenInformation()`, the `LookupAccountSidW()` resolution, and the recovered `ADMINISTRATOR` and `XDRAPRDWV010T` strings.

6.2 Process Elevation Check

CheckProcessElevation() (0x00492310) opens the current process's token again and queries it with GetTokenInformation() using TokenInformationClass = 0x14 (TokenElevation), returning TokenIsElevated = 1.

Per Microsoft's documented TOKEN_ELEVATION structure, this means the process token is running with administrative privileges. This was logged as "process is elevated: 1."



```

0049235D  pop ebx
0049235E  ret
0049235F  nop
00492360  lea eax, dword ptr ss:[esp+2C]
00492364  mov dword ptr ss:[esp+2C], 4
0049236C  mov dword ptr ss:[esp+4], 4
00492374  mov dword ptr ss:[esp+14], 14
0049237C  mov dword ptr ss:[esp+10], eax
00492380  lea eax, dword ptr ss:[esp+28]
00492384  mov dword ptr ss:[esp+8], eax
00492388  mov eax, dword ptr ss:[esp+24]
0049238C  mov dword ptr ss:[esp], eax
0049238F  call dword ptr ds:[<GetTokenInformation>]
00492395  sub esp, 14
00492398  test eax, eax
0049239A  cmovne ebx, dword ptr ss:[esp+28]
0049239F  jmp sample.492344
004923A1  jmp sample.492380
004923A3  nop
004923A4  nop
004923A5  nop
004923A6  nop
004923A7  nop
004923A8  nop

```

Registers:

```

EAX 00000001
EBX 00000000
ECX 02210000
EDX 00000000
ESP 007BFEA8
ESI ECCE136E
EDI 561C24DE
EIP 00492395 sample.00492395

```

Flags: 00000200
ZF 0 PF 0 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError: 00000000 (ERROR_SUCCESS)
LastStatus: 00000000 (STATUS_SUCCESS)

Default (tidcall) 5 Unlocked

1: [esp+4] 007BFEA8 007BFEA8
2: [esp+8] 004A56C7 sample.004A56C7
3: [esp+C] 007BF2D0 007BF2D0
4: [esp+10] 00000284 00000284
5: [esp+14] 00000001 00000001

Address Hex
007BF2E8 01 00 00 00 04 00 00 00 ED 54 06 0E FE FF FF FFT...b99?
007BF308 A4 2F 9F 00 C3 E2 45 00 38 94 9F 00 38 94 9F 00 R/.A.E.8...8...
007BF318 6E 13 CC EE DE 24 1C 56 FE FF FF 01 E8 F2 78 00 n.1fD\$.Vb9y..eb{.
007BF328 03 5D 5A 77 B8 F4 78 00 00 00 00 00 AB D1 3E 2C }Jw 0{...%N...
007BF338 FC FF FF FF C8 F4 78 00 02 00 00 40 00 00 00 00 b9y0{.b.#...
007BF348 D3 DF 23 5B 00 00 00 D8 00 00 00 00 00 A7 00 08{...0....\$.
007BF358 00 00 00 00 E8 F4 78 00 03 5D 5A 77 D0 00 00 00 ...eb{..}zw0...
007BF368 F3 DF 23 5B 00 00 00 D8 00 00 00 00 00 A7 00 08{D...0....\$.

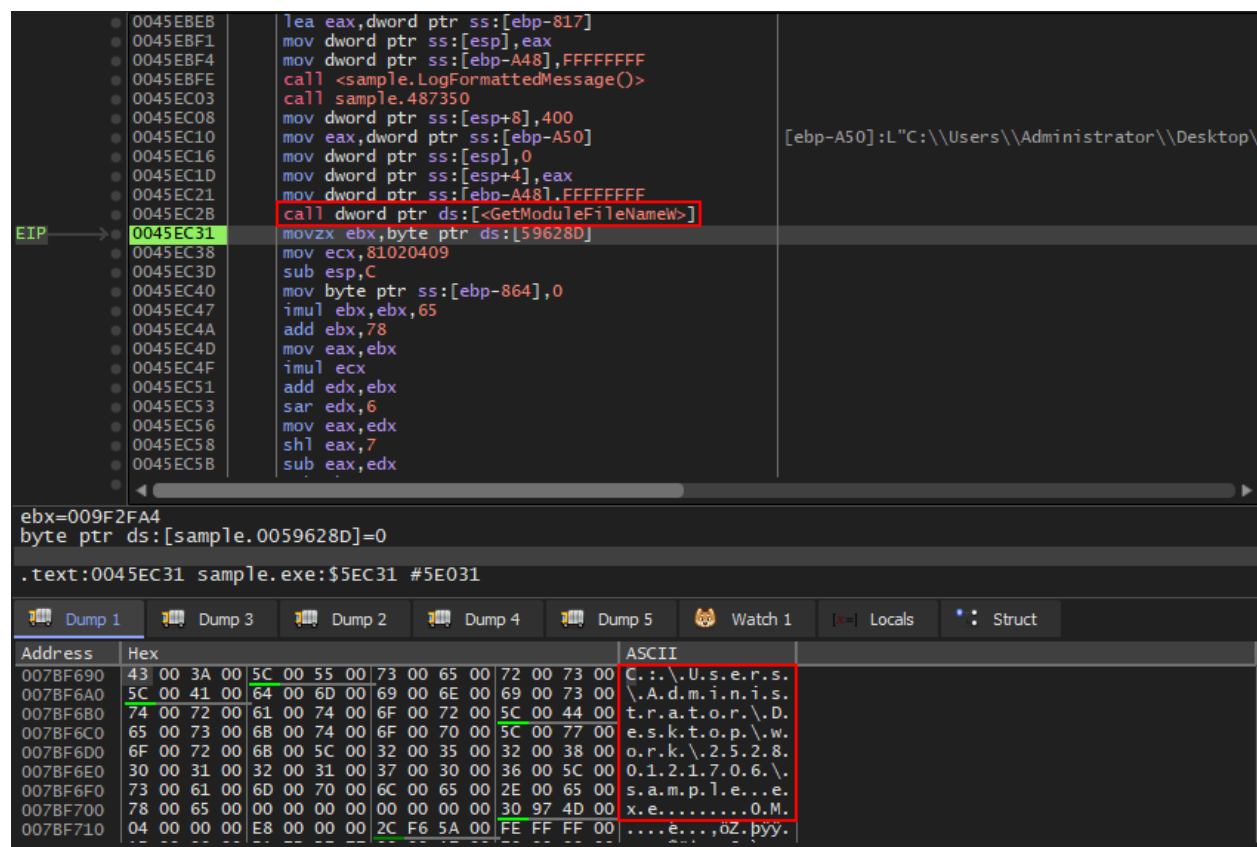
FIGURE 11. GetTokenInformation(TokenElevation) returning TokenIsElevated = 1.

6.3 Executable Path Resolution and Architecture Check

GetModuleFileNameW() returns the running executable's own path, logged as "Current executable path: C:\Users\Administrator\Desktop\work\25280121706\sample.exe," then split into a directory and filename by FindLastPathSeparator() (0x004F6110).

That directory path is then passed to a mode-check routine at 0x0048AC30, which reads a single byte at 0x005A7BC0 and branches on its value. In the observed run, neither branch condition was met. This routine also decrypts the strings truesight.sys and rentdrv2.sys, both known vulnerable drivers associated with Bring Your Own Vulnerable Driver (BYOVD) activity.

Finally, GetCurrentProcess() returns the pseudo-handle 0xFFFFFFFF, and passing it to IsWow64Process() returns 1 - confirming this 32-bit sample is running on a 64-bit Windows host via WOW64.



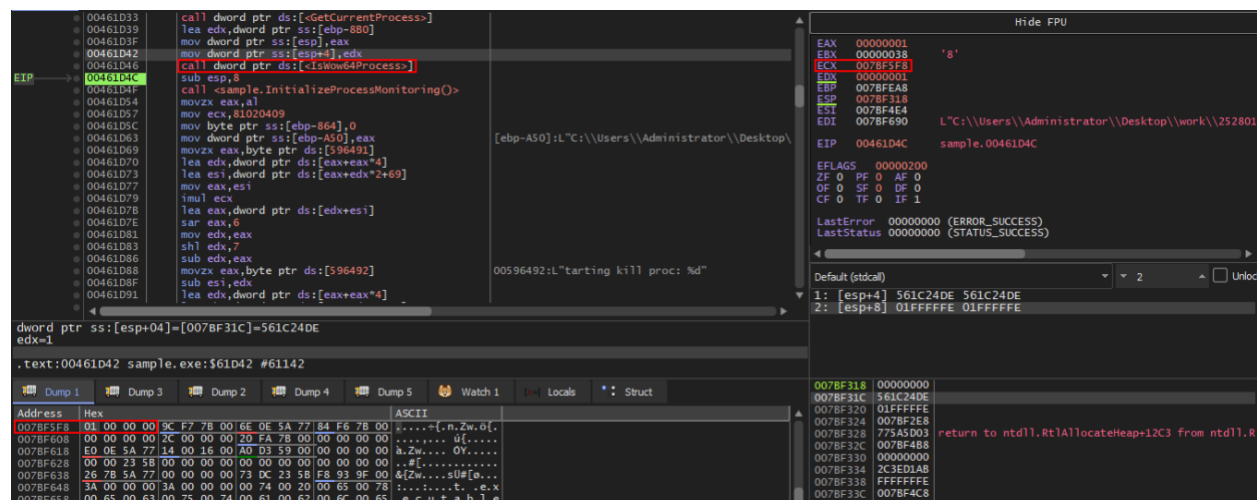
```

0045EBE8 lea eax,dword ptr ss:[ebp-817]
0045EBF1 mov dword ptr ss:[esp],eax
0045EBF4 mov dword ptr ss:[ebp-A48],FFFFFFFF
0045EBFE call <sample.LogFormattedMessage>
0045EC03 call sample.487350
0045EC08 mov dword ptr ss:[esp+8],400
0045EC10 mov eax,dword ptr ss:[ebp-A50]
0045EC16 mov dword ptr ss:[esp],0
0045EC1D mov dword ptr ss:[esp+4],eax
0045EC21 mov dword ptr ss:[ebp-A48],FFFFFFFF
0045EC28 call dword ptr ds:[<GetModuleFileNameW>]
EIP -> 0045EC31 movzx ebx,byte ptr ds:[59628D]
0045EC38 mov ecx,81020409
0045EC3D sub esp,C
0045EC40 mov byte ptr ss:[ebp-864],0
0045EC47 imul ebx,ebx,65
0045EC4A add ebx,78
0045EC4D mov eax,ebx
0045EC4F imul ecx
0045EC51 add edx,ebx
0045EC53 sar edx,6
0045EC56 mov eax,edx
0045EC58 shl eax,7
0045EC5B sub eax,edx

```

ebx=009F2FA4
byte ptr ds:[sample.0059628D]=0
.text:0045EC31 sample.exe:\$5EC31 #5E031

Address	Hex	ASCII
007BF690	43 00 3A 00 5C 00 55 00 73 00 65 00 72 00 73 00	C.:.\\U.s.e.r.s.
007BF6A0	5C 00 41 00 64 00 6D 00 69 00 6E 00 69 00 73 00	\\A.d.m.i.n.i.s.
007BF6B0	74 00 72 00 61 00 74 00 6F 00 72 00 5C 00 44 00	t.r.a.t.o.r.\\D.
007BF6C0	65 00 73 00 6B 00 74 00 6F 00 70 00 5C 00 77 00	e.s.k.t.o.p.\\w.
007BF6D0	6F 00 72 00 68 00 5C 00 32 00 35 00 32 00 38 00	o.r.k.\\2.5.2.8.
007BF6E0	30 00 31 00 32 00 31 00 37 00 30 00 36 00 5C 00	0.1.2.1.7.0.6.\\
007BF6F0	73 00 61 00 6D 00 70 00 6C 00 65 00 2E 00 65 00	s.a.m.p.l.e.e.
007BF700	78 00 65 00 00 00 00 00 00 00 00 30 97 4D 00	x.e.....0.M.
007BF710	04 00 00 00 E8 00 00 00 2C F6 5A 00 FE FF FF 00	e....o.z.pyy.



```

00461D33 call dword ptr ds:[<GetCurrentProcess>]
00461D39 lea edx,dword ptr ss:[ebp-880]
00461D3F mov dword ptr ss:[esp],edx
00461D42 call dword ptr ds:[<IsWow64Process>]
EIP -> 00461D4C sub esp,8
00461D51 call <sample.InitializeProcessMonitoring>
00461D54 movzx eax,al
00461D57 mov ecx,81020409
00461D5C mov byte ptr ss:[ebp-864],0
00461D63 mov dword ptr ss:[ebp-A50],eax
00461D69 movzx eax,byte ptr ds:[596491]
00461D70 lea edx,dword ptr ds:[eax+eax*4]
00461D73 lea esi,dword ptr ds:[eax+edx*2+69]
00461D77 mov eax,esi
00461D79 imul ecx
00461D7B lea eax,dword ptr ds:[edx+esi]
00461D7E sar eax,6
00461D81 mov edx,edx
00461D83 shl edx,7
00461D86 sub edx,eax
00461D88 movzx eax,byte ptr ds:[596492]
00461D8F sub esi,edx
00461D91 lea edx,dword ptr ds:[eax+eax*4]

```

dword ptr ss:[esp+04]-[007BF31C]=561C24DE
edx=1
.text:00461D42 sample.exe:\$61D42 #61142

Address	Hex	ASCII
007BF5E8	01 00 00 00 5C F7 78 00 6E 0E 5A 77 84 F6 78 00	n.zw.o[.
007BF608	00 00 00 00 2C 00 00 00 20 FA 78 00 00 00 00 00	u[.....
007BF618	E0 0E 5A 77 14 00 16 00 A0 D3 59 00 00 00 00 00	h.zw....OY.....
007BF628	00 00 23 58 00 00 00 00 00 00 00 00 00 00 00	..[.....
007BF638	26 7B 5A 77 00 00 00 00 72 DC 23 58 F8 93 9E 00	h.zw....suf[0..
007BF648	3A 00 00 00 3A 00 00 00 00 74 00 20 00 65 00 78	t..e.x
007BF658	00 65 00 63 00 75 00 74 00 61 00 62 00 6C 00 65	..e.c.u.t.a.b.l.e

FIGURE 12. *GetModuleFileNameW()* recovering the executable path, followed by *IsWow64Process()* returning 1.

6.4 Explorer Process Identification

Separately, *EnumerateExplorerProcess()* (0x0046D980) performs a full *CreateToolhelp32Snapshot()/Process32FirstW()/Process32NextW()* pass over the running process list, decrypts the comparison string *explorer.exe*, and stores the matching PID. No later use of the stored PID

was identified. The desktop-customization mechanism in Section 12 instead relies on registry changes and SystemParametersInfoW().

Function	Address	Role
CollectUserAndHostInformation()	0x0048CAA0	Processes current-token SID information and recovers account/host strings
CheckProcessElevation()	0x00492310	Confirms the process token is elevated
FindLastPathSeparator()	0x004F6110	Splits the executable path into directory/filename
Driver-related mode-check routine	0x0048AC30	Reads a mode byte and operates on driver-related strings when it matches
EnumerateExplorerProcess()	0x0046D980	Locates and stores the explorer.exe PID

7. Process Discovery and Termination

With reconnaissance done, DragonForce starts two background threads that repeatedly scan the running-process list for names in the configured priority list (Section 4.3) and attempt to terminate matching processes.

7.1 Setting Up the Monitoring Threads

InitializeProcessMonitoring() (0x00479B30) first resolves NtQuerySystemInformation, logs "Starting proc:", and creates two threads:

Argument	Thread 1	Thread 2
lpStartAddress	0x0047DE00	0x0047E8A0
lpParameter	NULL	NULL
dwCreationFlags	0 (runs immediately)	0 (runs immediately)

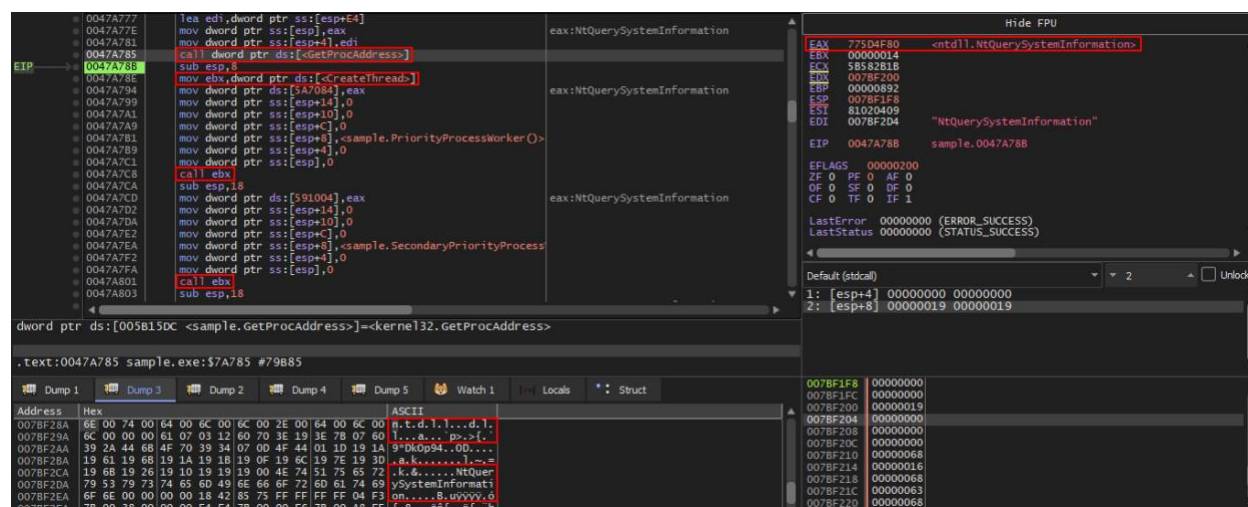


FIGURE 13. *InitializeProcessMonitoring()* resolving *NtQuerySystemInformation* and creating the two monitoring workers.

7.2 Matching and Terminating Processes

The first worker, *PriorityProcessWorker()* (0x0047DE00), logs "Starting priority proc," then walks the priority-process list one entry at a time, passing each name to *EnumerateAndMatchProcess()* (0x0047DDB0), which calls the resolved *NtQuerySystemInformation()* twice, walks the returned process list, and compares names character-by-character using the string-recovery routine.

When a match is found, the PID is logged and passed to *TerminateTargetProcess()* (0x00475EC0), which calls *OpenProcess()* and, on success, *TerminateProcess()*. Across two observed executions, the same two outcomes recur:

Target	Run A Result	Run B Result
MsMpEng.exe	PID 5096, <i>OpenProcess()</i> failed - <i>ERROR_ACCESS_DENIED</i>	PID 3120, <i>OpenProcess()</i> failed - <i>ERROR_ACCESS_DENIED</i>
Firefox.exe	PID 10144, opened and terminated successfully	PID 9128, opened and terminated successfully

After a full pass through the priority list, the worker sleeps for 0xFA (250 ms) and repeats.

```
"[20:39:45] [Th:2764] Starting priority proc\r\n"
"[20:39:58] [Th:2764] [+] Process 'MsMpEng.exe' PID: 3120\r\n"
"[20:41:07] [Th:2764] [-] OpenProcess failed: MsMpEng.exe\r\n"
"[20:41:23] [Th:2764] [+] Process 'firefox.exe' PID: 9128\r\n"
"[20:41:52] [Th:2764] [+] Terminating PID: 9128 for process 'firefox.exe'\r\n"
"[20:42:03] [Th:2764] [+] Process terminated: firefox.exe\r\n"
```

FIGURE 14. Recovered Log: The repeated MsMpEng.exe access-denied result alongside a successful Firefox.exe termination.

The second thread, SecondaryPriorityProcessWorker() (0x0047E8A0), advances through entries with an 0x18-byte stride and calls the same EnumerateAndMatchProcess()/TerminateTargetProcess() logic as the first worker. The static code path is capable of terminating a matched process exactly as the first worker does.

7.3 Mutex Creation and Main-Thread Continuation

After both threads are created, the main thread logs "Starting kill proc: 1," then recovers a mutex name via the string-recovery routine and calls CreateMutex() with bInitialOwner = TRUE and lpName = "hsfjuukjzloqu28oajh727190." The main thread then calls WaitForSingleObject() with a zero timeout, which tests the object's state and returns immediately - a one-shot state check, not a delay for the worker threads to catch up.

Function	Address	Role
InitializeProcessMonitoring()	0x00479B30	Resolves NtQuerySystemInformation, creates the two worker threads
PriorityProcessWorker()	0x0047DE00	Walks priority list, terminates matches, sleeps 250ms, repeats
SecondaryPriorityProcessWorker()	0x0047E8A0	Same static logic. Only reached its sleep loop in the observed run
EnumerateAndMatchProcess()	0x0047DDB0	Compares a target name against the live process list
TerminateTargetProcess()	0x00475EC0	Opens and terminates a matched process

8. Anti-Recovery: Shadow Copy Deletion

Before target-discovery and file-processing, DragonForce queries Windows' built-in Volume Shadow Copy mechanism and prepares a command to delete individual shadow copies.

8.1 COM/WMI Initialization and Locator Creation

ManageShadowCopies() (0x00409280) reaches the shadow-copy query through the standard WMI client sequence documented by Microsoft. After a compiler-generated stack-probing routine, the function calls CoInitializeEx() and CoInitializeSecurity() to bring up COM, then CoCreateInstance() for CLSID_WbemLocator.

```
CoInitializeEx()  
  |  
  v  
CoInitializeSecurity()  
  |  
  v  
CoCreateInstance(CLSID_WbemLocator) --> IWbemLocator interface
```

Between creating the locator and connecting to a namespace, DragonForce calls GetNativeSystemInfo() and compares the processor architecture field against 9 (PROCESSOR_ARCHITECTURE_AMD64), confirming the host is a 64-bit machine.

8.2 Connecting, Querying, and Deleting Shadow Copies

Using the decrypted string ROOT\CIMV2, the locator's ConnectServer() method returns an IWbemServices interface, and CoSetProxyBlanket() configures its security settings. DragonForce then calls ExecQuery() with the WQL query:

```
SELECT * FROM Win32_ShadowCopy
```

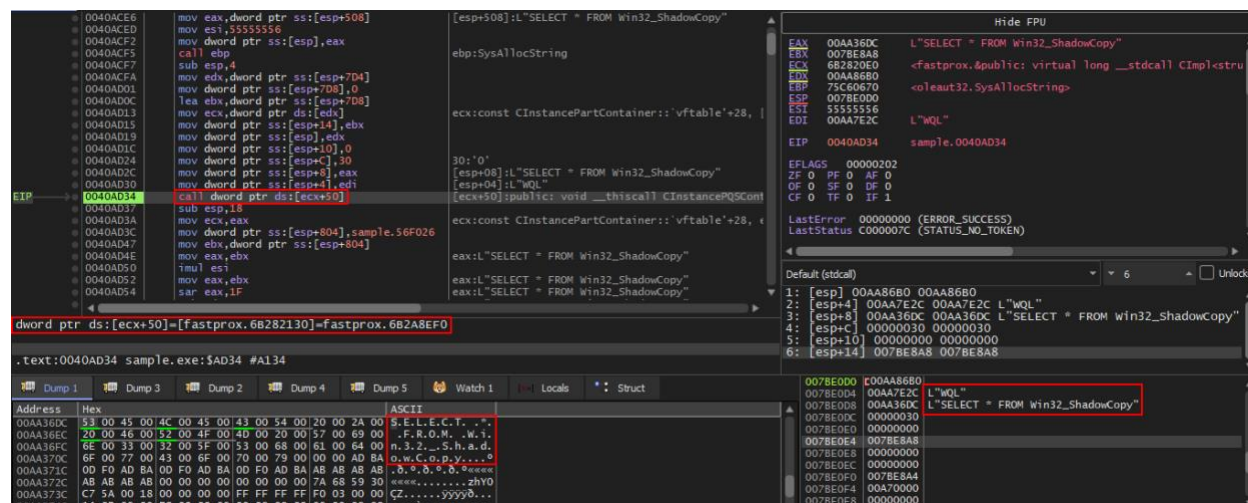


FIGURE 15. ExecQuery() issued with SELECT * FROM Win32_ShadowCopy

That ID is used to build the following command string:

```
cmd.exe /c C:\Windows\System32\wbem\WMIC.exe shadowcopy where "ID='%s'" delete
```

The sample invokes WMIC.exe to delete the volume shadow copies enumerated through the Win32_ShadowCopy WQL query. Dynamic tracing confirmed that execution completes the COM/WMI query phase, populates the command template with the retrieved shadow copy identifier, and executes the deletion command to eliminate local recovery points before file processing begins. The routine then releases its WMI/COM interface pointers and finalizes COM with CoUninitialize().

Function	Address	Role
ManageShadowCopies()	0x00409280	Drives the COM/WMI shadow-copy query and deletion-command construction
StackProbeHelper()	0x004DA4E0	Compiler-generated stack probe

9. Local and Network Target Discovery

DragonForce builds its target list from the local machine's logical drives and from hosts represented in the local ARP cache whose SMB shares it can enumerate. Both target classes feed the encryption workers (Section 10) through two separate task queues.

9.1 Logical Drive Enumeration and Task Queue

EnumerateLogicalDrives() (0x0044CEE0) calls GetLogicalDriveStringsW() twice and in the analyzed run recovered two drives (C:\ and D:\). Each path is handed to PutTask() (0x00486BF0), which inserts the path into a shared, critical-section-protected task structure for the encryption workers to consume.

Encryption-worker activity begins immediately after the C:\ task is published, before D:\ is even inserted. Local-drive discovery and file encryption therefore proceed concurrently rather than as separate phases.

```
"[21:26:18] [Th:1280] Found 2 drives: \r\n"
"[21:27:15] [Th:1280] C:\\\\r\n"
"[21:27:26] [Th:1280] D:\\\\r\n"
"[21:28:29] [Th:1280] PutTask: C:\\\\r\n"
"[21:29:09] [Th:1280] PutTask: D:\\\\r\n"
```

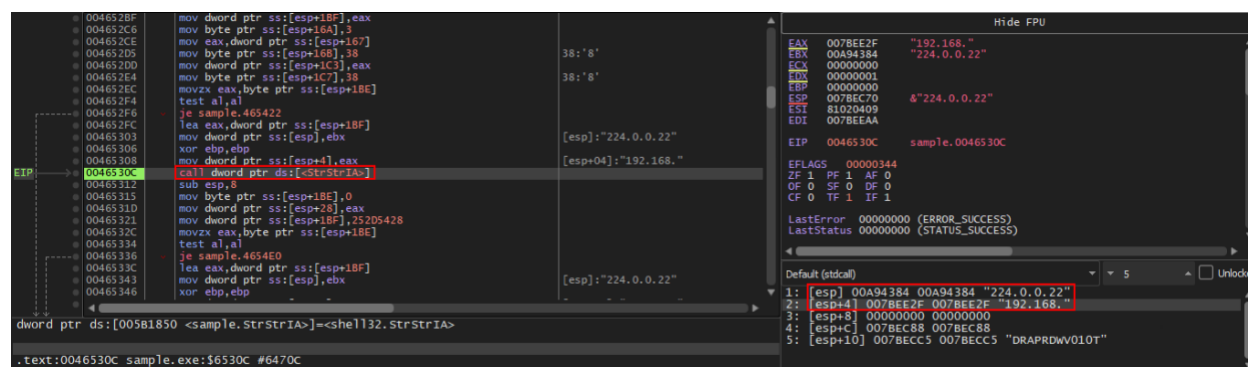
FIGURE 16. Recovered log: Found 2 drives, C:\, D:\, and the corresponding PutTask entries.

9.2 Internal Network Discovery

A separate function (0x00469CF0) drives everything network-related, starting with WSASStartup(). The actual discovery mechanism is passive, not an active network scan: EnumerateInternalIPs() (0x00463B80) calls GetIpNetTable() - the Windows ARP cache - then walks every entry, testing each candidate address against four decrypted prefix strings:

Prefix Checked	Range Represented
172.	RFC 1918 private range (172.16.0.0/12, checked loosely by prefix)
192.168.	RFC 1918 private range (192.168.0.0/16)
10.	RFC 1918 private range (10.0.0.0/8)
169.	Link-local range (169.254.0.0/16)

An ARP entry for the multicast address 224.0.0.22 was observed being tested against all four prefixes and matching none. The next entry, 192.168.9.1, matched the 192.168. prefix and was added to the candidate list. The function reads the existing ARP cache rather than actively sweeping the subnet.



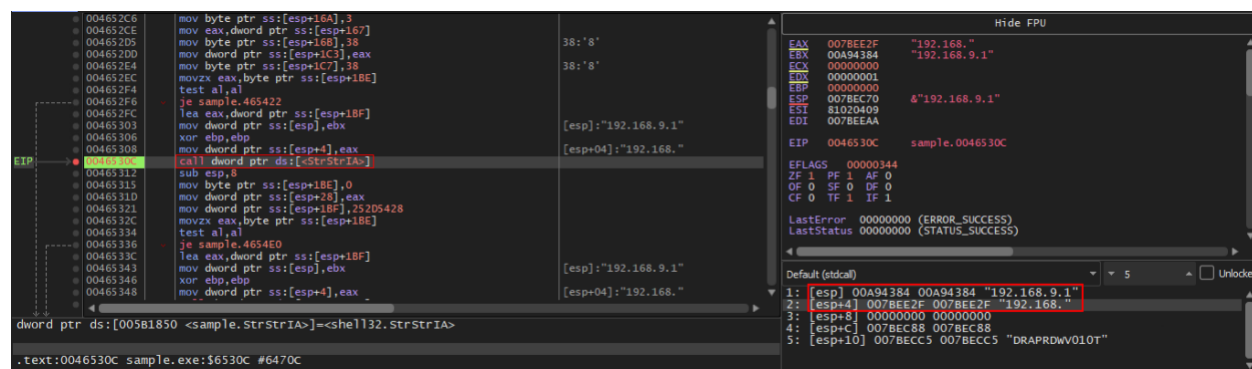


FIGURE 17. 224.0.0.22 rejected and 192.168.9.1 accepted on the 192.168. prefix check.

9.3 SMB Share Enumeration and Propagation

For each internal IP address, EnumerateAndAppendShares() (0x00466760) calls NetShareEnum() and processes the returned share names. Most discovered targets were administrative C\$ shares, but the debugger-captured log also records \\192.168.9.114\share and \\192.168.9.114\Users reaching the task-publication path, demonstrating that non-C\$ shares can also be queued.

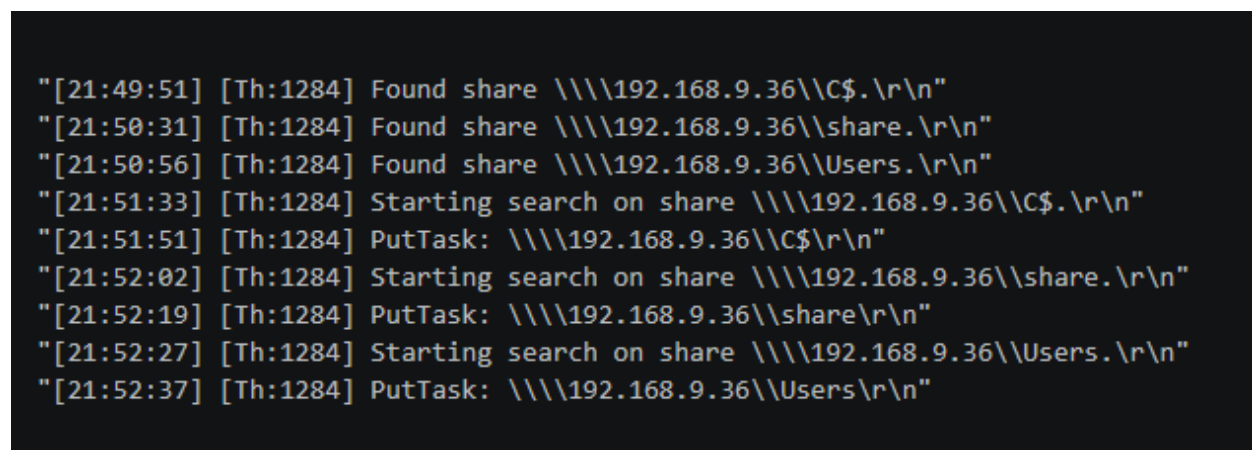


FIGURE 18. Recovered log: Found share, Starting search on share, and PutTask entries for both C\$ and non-C\$ shares.

Across the full run, the debugger-captured log recorded seventeen share discoveries across more than a dozen distinct internal IP addresses:

Found share \\192.168.9.36\C\$	Found share \\192.168.9.114\C\$
Found share \\192.168.9.36\Users	Found share \\192.168.9.114\share
Found share \\192.168.9.35\C\$	Found share \\192.168.9.114\Users
Found share \\192.168.9.110\C\$	Found share \\192.168.9.151\C\$
Found share \\192.168.9.111\C\$	Found share \\192.168.9.154\C\$
Found share \\192.168.9.115\C\$	Found share \\192.168.9.180\C\$
Found share \\192.168.9.112\C\$	Found share \\192.168.9.181\C\$
Found share \\192.168.9.116\C\$	Found share \\192.168.9.182\C\$
	Found share \\192.168.9.248\C\$

Function	Address	Role
EnumerateLogicalDrives()	0x0044CEE0	Enumerates local drives, queues each via PutTask()
PutTask()	0x00486BF0	Shared task-publication function for drives and shares
EnumerateInternalIPs()	0x00463B80	Walks the ARP cache, filters candidate addresses
EnumerateAndAppendShares()	0x00466760	Runs NetShareEnum(), builds UNC paths, calls PutTask()

10. Concurrent File Encryption

This is the section that does the actual damage. One detail matters more than any other here: DragonForce does **not** use one encryption key for everything. Configuration and log handling rely on recoverable state material, while per-file key material is generated at runtime and then processed with the embedded RSA public key.

10.1 Encryption Worker Pool and Cryptographic Setup

InitializeEncryptionWorkers() (0x00486B60) is called twice from the main thread, creating eight worker threads each time - sixteen total, all running EncryptionWorker() (0x004866F0).

Each worker's first action is AcquireCryptoProviderContext() (0x00485BE0), which calls CryptAcquireContextA() to get a CryptoAPI provider handle. Each worker then calls CryptImportKey(), loading an embedded key BLOB from 0x00591020:

```
CryptImportKey(hProv, pbData=0x00591020, dwDataLen=0x214, hPubKey=0, dwFlags=0)
```

```
0x00591020: 06 02 00 00 00 A4 00 00 52 53 41 31 00 10 00 00
              01 00 01 00 BD F9 1D 9A 76 12 2A C1 13 1A 8B B5 ...
```

The RSA1 marker identifies this as a public RSA key BLOB. The 0x214-byte structure contains a 12-byte RSAPUBKEY header followed by a 512-byte modulus, establishing a 4096-bit RSA key with public exponent 0x00010001 (65537). The corresponding RSA private key is not present in the sample.

 Dump 3

Address	Hex	ASCII
00591020	06 02 00 00 00 A4 00 00 52 53 41 31 00 10 00 00	RSA1....
00591030	01 00 01 00 BD F9 1D 9A 76 12 2A C1 13 1A 8B B5	%ù..v.*A...μ
00591040	E9 10 D9 1A DF EF 9B 78 28 CB 0B 8E F1 B8 4B 96	é.Ü.ßî.x(E...ñ.K.
00591050	7F 07 C9 8B C0 85 AF 62 BC 8C 8D A3 6B 90 3F FE	..E.A.▯%...fk.7b
00591060	BD D6 48 9B 78 BD F4 A3 E6 81 7A 9F 22 D0 B5 85	½ÖH.x%ôfæ.z."ðμ.
00591070	76 9B F1 67 B2 56 95 E3 12 FF CB 5C A5 4A 08 A8	v.ñg°V.ä.ÿE°ÿJ. "
00591080	A7 EA 51 EA D3 83 1E 11 A6 2B 27 84 FE A5 73 4C	§ëQëÖ...!+'..b°sL
00591090	1C A8 DA 06 C3 46 8A 35 A1 C1 6A CE 70 95 31 45	..Ü.AF.5;AjIp.1E
005910A0	29 61 28 2D DC B6 3E D6 31 29 E5 57 04 E2 86 37	)a(-Ü¶>Ö1)âw.â.7
005910B0	58 83 BF FD 6C 25 72 CD DA 9F 00 BF 57 E0 57 AB	X.¿ýl°rIU...¿wâw«
005910C0	B3 2E E8 B5 FC ED 9E E7 26 36 B8 C0 5A AD 8A E0	°.èµüí.ç&6.AZ...à
005910D0	BB 84 44 1F D1 E9 CE 6B EB 1B 28 F9 41 93 D8 97	».D.NéÍkë.(uA.Ø.
005910E0	86 DB 11 D9 4B B7 B5 73 9C C2 66 9D 28 EC 69 3D	.0.UK·µs.Af.(îi=
005910F0	A4 16 7B F8 47 2F 64 78 DD FF 25 23 41 8C F9 FF	¤.{øG/dxÿÿ%#A.ùÿ
00591100	06 FD 67 F0 0E 72 FF 7B 42 D8 67 B3 82 22 C5 F5	.ýgð.rÿ{B0g°. "Að
00591110	61 4A F1 1C A8 7E 9B 05 7D BA BA 03 19 ED AC F7	aJñ.~...}°°..í~
00591120	0F AF AA 6F 09 EA AF 8E 32 C0 B8 7D 1D 45 98 F8	..°o.ê~.2A.}.E.ø
00591130	E8 E3 88 6C 03 B4 AE F7 BA 51 89 12 E3 7A FB 09	èä.1.°÷°Q...äzù.
00591140	A2 79 A7 64 96 59 E1 1E AE 97 27 3E 6B 8F 53 BF	çy§d.Yá.°. '>k.S¿
00591150	61 7C 59 E6 8B 16 C0 B6 FE F0 55 B4 91 8F B5 4C	a Yæ..A¶bðU'..µL
00591160	62 80 F9 08 54 99 98 06 65 66 1C C0 88 DB 8B D0	b.ù.T...ef.A.0.ð
00591170	4F B8 8C 7D 8F 44 ED 26 D8 D4 A2 64 6E 7E 85 7F	O».}.Dí&00cðn~..
00591180	F8 23 58 AF 4F EA 8F B2 2C 97 C3 2F F9 02 2F 04	ø#X°Oè.°,..A/ù./.
00591190	F8 8E 21 F5 D6 C4 3F F5 D4 5F 56 3A FD 92 8D 38	û. !ðÖA?ðÖ_V:ý..8
005911A0	23 68 A9 27 D3 49 F1 4A 64 41 30 C1 80 AE EC 3F	#hø'OIñJdA0A.øi?
005911B0	6B 6D D0 3D 71 8A E5 4B 6F E6 02 28 71 72 84 8C	kmð=q.âKøæ.(qr..
005911C0	3B 55 F2 F3 8A 13 C4 32 7F 9D EC BA 3A F6 94 F7	;Uòò...Ä2...ì°:ö.÷
005911D0	9D 62 7C 44 C1 99 F6 C7 56 5C B1 64 4F D9 7C 2C	.b DA.öÇV\±d0U .
005911E0	5A 8D C9 FE CD B1 7B A5 75 6A 44 AA A6 BD 48 33	Z.EbI±{¥ujD±!½H3
005911F0	6F 93 02 33 10 64 B3 AF 61 EE BA 3D 05 5B BC 2D	o...3.d°-aî°=. [¼-
00591200	9B E1 3D EA 4D 12 60 51 10 81 87 CC 8E C1 F5 2F	.â=èM.°Q...I.Að/
00591210	05 92 82 21 04 B4 FA C1 2D 6D 2D 5D 34 00 89 F6	...!.°úA-m-]4...ö
00591220	DC D5 3B E9 90 6B F5 C4 CA 55 2A 01 9C 08 0E F3	U0;é.kµÆEU*....ó
00591230	CF 58 5C D9 90 E6 FD 00 51 B0 8E 02 03 88 3A BF	IX\Ü.æý.Q°....:¿
00591240	2D D2 7B C6 F7 6D CB 83 5E B6 C9 11 5A A4 08 F0	-Ö{Æ÷mE.¶¶E.Z¤.ð
00591250	43 3F 3A A6 9E 9F D3 F2 8C 46 EC 47 A8 4A FB D1	C?: .°ò.FiG`JúÑ
00591260	E9 D5 30 56 69 13 81 C6 72 71 F8 3E EE 03 B3 E2	é00Vi..Ærqø>î.°â
00591270	9A DC E3 49 E8 58 64 16 8A 3D 39 07 50 AC EB 44	.ÜäIèXd...=9.P-èD
00591280	3E 9B 01 53 0E 58 54 5A 92 F4 61 E7 CD 51 F8 49	>...S.XTZ.ðaçIQøI
00591290	69 80 78 E6 49 2B A3 C0 A3 F6 4F 8A 55 D0 60 B4	i.xæI+£AföO.Uð`
005912A0	B1 6D 61 28 CB 67 00 EB BF 15 CE DA B6 69 3C B9	±ma(Eg.ë¿.IU¶i<'
005912B0	7F F0 EF 9F CB 6B 1B 99 AB 26 CF BF 55 AC 65 DC	.ðî.Ek...«&I¿U-eÜ
005912C0	C0 25 B6 8F 9A 74 68 91 7B 15 20 74 DE 11 1D 99	A%¶...th.{. tð...
005912D0	68 31 B7 72 98 3A 74 42 A2 BE 2C 12 E3 36 83 91	h1.r..:tBç%,.ä6..
005912E0	D9 A6 61 CB 0A C8 FA 8B E6 4B 50 1D DD 63 B5 F3	Ü!aE.Éú.æKP.ÿcµó
005912F0	81 0D 2C CA 68 C9 3B 63 C7 8F 83 98 79 B5 5D B3	...èhE;cÇ...ÿµ)°
00591300	7B A0 A3 6B C7 04 98 64 0A 9C 71 8E D5 4C CB CF	{ fkcÇ...d...q.ÖLEI
00591310	D9 6B BD 17 1B BD D8 C1 3A 89 BB 43 7F 38 EC 0D	Ük½..½0A:..»C.8ì.
00591320	B5 9A EA B5 4C 38 83 C5 7B 19 1A 8A F4 5E DB CD	µ.èµL8.A{...ð^OI
00591330	FF A8 F4 46 4D 05 33 59 AA 6E 8B 6B 56 5C B2 C3	ÿ°ðFM.3Y°n.kV\°A
00591340	93 7E 27 B2 0F BD 2E DE 45 2A DB 6D 9D 86 D4 6B	.~°°.½.ðE°0m...0k
00591350	73 FB 65 21 43 1B 55 9C 9C 7F 21 04 62 0D EA CD	sûe!C.U...!.b.èI

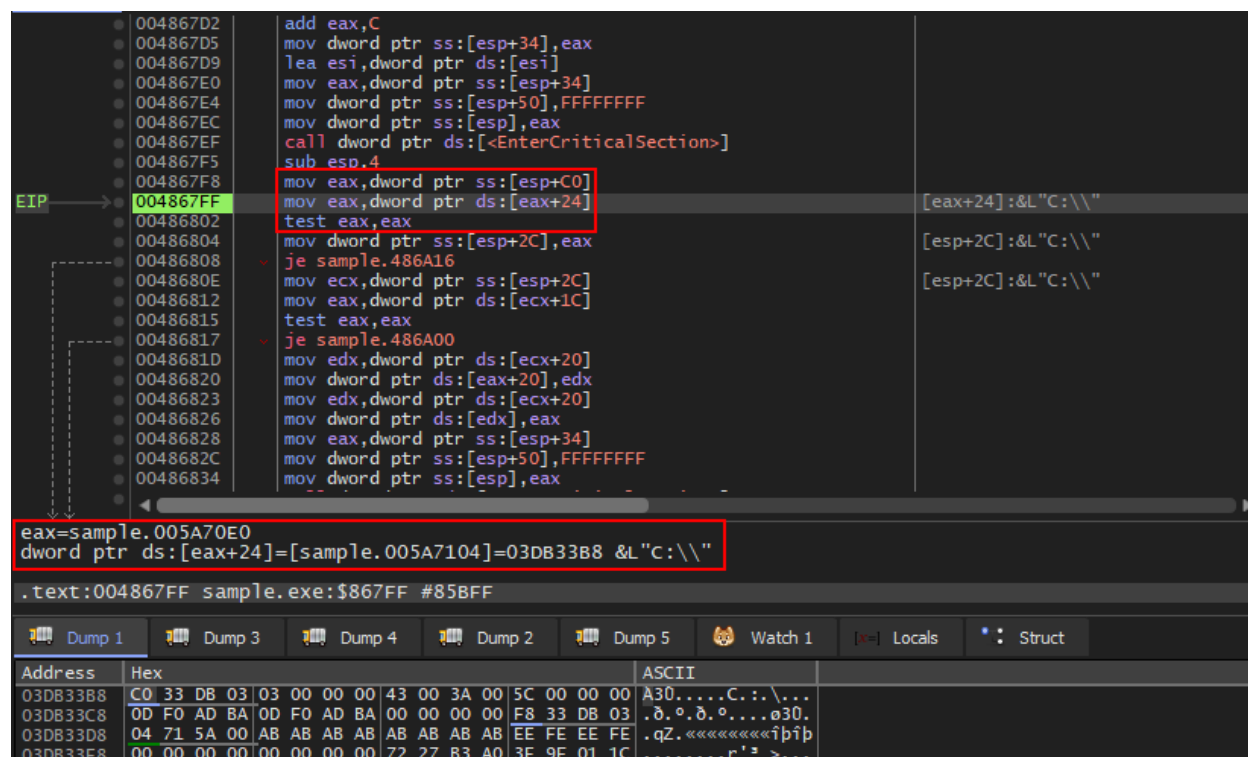
FIGURE 19. Memory view of the embedded RSA PUBLICKEYBLOB at 0x00591020.

10.2 Task Queue Architecture: Two Separate Queues

Each worker enters a critical section and checks a shared wrapper structure for an available task. The first batch checks a wrapper at 0x005A70E0, whose +0x24 field points to the local-drive task list head

(Section 9.1). The second batch checks a wrapper at 0x005A70A0, whose +0x24 field points to the network-share task list head (Section 9.3).

Worker Batch	Wrapper Structure	+0x24 Points to	Feeds From
First 8 (local drives)	0x005A70E0 +0x24	0x005A7104	Local drives (9.1)
Second 8 (network shares)	0x005A70A0 +0x24	0x005A70C4	Network shares (9.3)



```

004867D2 add eax,C
004867D5 mov dword ptr ss:[esp+34],eax
004867D9 lea esi,dword ptr ds:[esi]
004867E0 mov eax,dword ptr ss:[esp+34]
004867E4 mov dword ptr ss:[esp+50],FFFFFFFF
004867EC mov dword ptr ss:[esp],eax
004867EF call dword ptr ds:[<EnterCriticalSection>]
004867F5 sub esp,4
004867F8 mov eax,dword ptr ss:[esp+C0]
004867FF mov eax,dword ptr ds:[eax+24]
00486802 test eax,eax
00486804 mov dword ptr ss:[esp+2C],eax
00486808 je sample.486A16
0048680E mov ecx,dword ptr ss:[esp+2C]
00486812 mov eax,dword ptr ds:[ecx+1C]
00486815 test eax,eax
00486817 je sample.486A00
0048681D mov edx,dword ptr ds:[ecx+20]
00486820 mov dword ptr ds:[eax+20],edx
00486823 mov edx,dword ptr ds:[ecx+20]
00486826 mov dword ptr ds:[edx],eax
00486828 mov eax,dword ptr ss:[esp+34]
0048682C mov dword ptr ss:[esp+50],FFFFFFFF
00486834 mov dword ptr ss:[esp],eax

```

eax=sample.005A70E0
dword ptr ds:[eax+24]=[sample.005A7104]=03DB33B8 &L"C:\\"

.text:004867FF sample.exe:\$867FF #85BFF

Address	Hex	ASCII
03DB33B8	C0 33 DB 03 03 00 00 00 43 00 3A 00 5C 00 00 00	A30.....C.:.\...
03DB33C8	0D F0 AD BA 0D F0 AD BA 00 00 00 00 F8 33 DB 03	.d.°.d.°....ø30.
03DB33D8	04 71 5A 00 AB AB AB AB AB AB AB AB EE FE EE FE	.qZ.««««««««iþþþ
03DB33E8	00 00 00 00 00 00 00 00 72 27 B3 A0 3E 9F 01 1C	r's>...

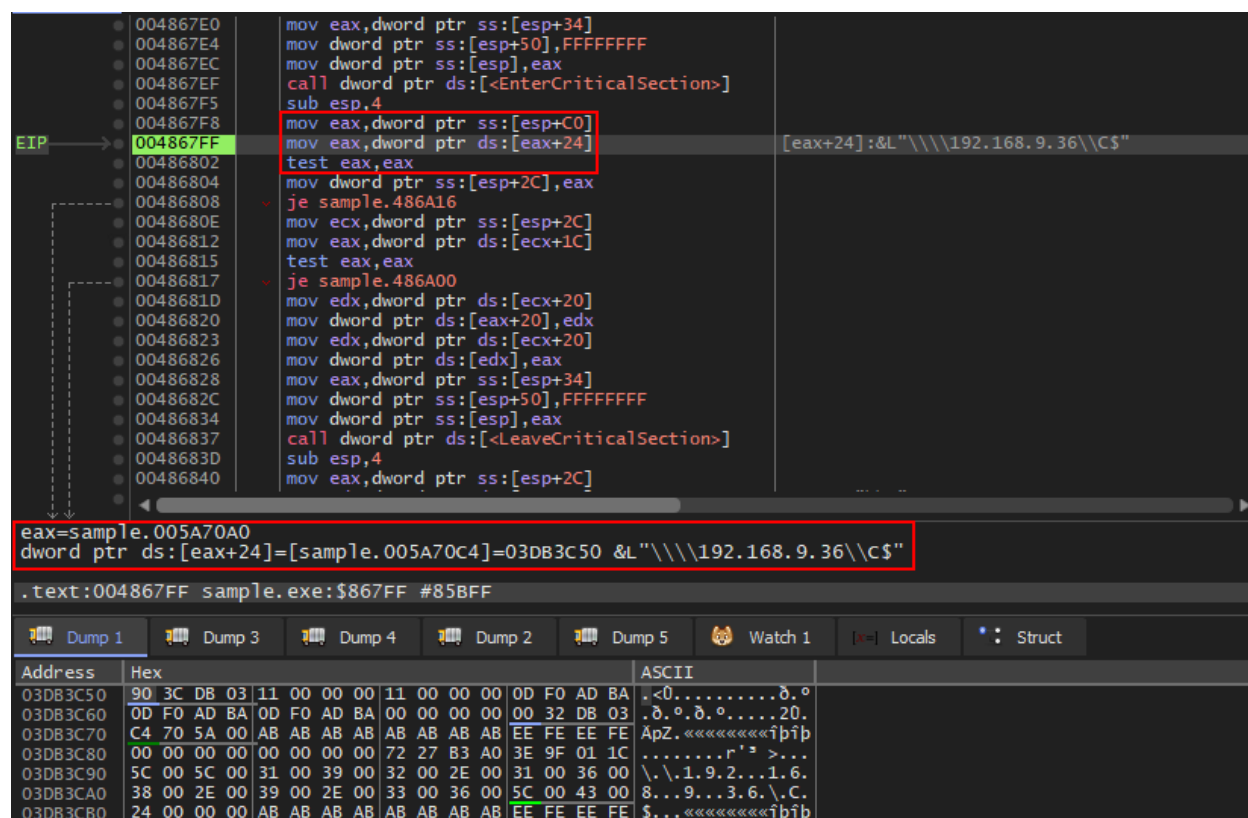


FIGURE 20. Debugger view of the two worker-wrapper structures and their +0x24 pointers to separate task-list heads.

When no task is available, a worker releases the lock and calls Sleep(0x1388) (5000 ms) before retrying. A second, distinct shutdown mechanism for permanently idle workers is documented in Section 11.1.

10.3 File Discovery and Filtering

Once a worker has a path, it walks the directory tree with FindFirstFileW()/FindNextFileW(). Two filters run before any file is touched:

CheckDirectoryExcluded() (0x0044DC10) compares each directory name against the whitelist from Section 4.3, and a filename/extension check compares the extension against the configured exclusion list and the filename against readme.txt.

Before reaching the configured checks, DragonForce also compares each filename against a second, hardcoded list of well over 200 extensions at 0x0042E510 - virtual machine/hypervisor disk formats (.vmdk, .vhd, .vhdx, .vmx, .vmem, .qcow2, .avhd, .vmcx, .nvram, .raw, .bin, .iso) and database formats (.4dd, .4dl, .accdb, .mdb, .sql, .sqlite, .fdb, .dbf) by taking jump to another function. Matching files are routed through DragonForce's partial-encryption path, encrypting 20% of the file content per other_encrypt_chunk_percent.

 Dump 5

Address	Hex	ASCII
005958DA	76 00 64 00 69 00 00 00 2E 00 76 00 68 00 64 00	v.d.i.....v.h.d.
005958EA	00 00 2E 00 76 00 6D 00 64 00 68 00 00 00 2E 00	v.m.d.k....
005958FA	70 00 76 00 6D 00 00 00 2E 00 76 00 6D 00 65 00	p.v.m.....v.m.e.
0059590A	6D 00 00 00 2E 00 76 00 6D 00 73 00 6E 00 00 00	m.....v.m.s.n...
0059591A	2E 00 76 00 6D 00 73 00 64 00 00 00 2E 00 6E 00	..v.m.s.d.....n.
0059592A	76 00 72 00 61 00 6D 00 00 00 2E 00 76 00 6D 00	v.r.a.m.....v.m.
0059593A	78 00 00 00 2E 00 72 00 61 00 77 00 00 00 2E 00	x.....r.a.w.....
0059594A	71 00 63 00 6F 00 77 00 32 00 00 00 2E 00 73 00	q.c.o.w.2.....s.
0059595A	75 00 62 00 76 00 6F 00 6C 00 00 00 2E 00 62 00	u.b.v.o.l.....b.
0059596A	69 00 6E 00 00 00 2E 00 76 00 73 00 76 00 00 00	i.n.....v.s.v...
0059597A	2E 00 61 00 76 00 68 00 64 00 00 00 2E 00 76 00	..a.v.h.d.....v.
0059598A	6D 00 72 00 73 00 00 00 2E 00 76 00 68 00 64 00	m.r.s.....v.h.d.
0059599A	78 00 00 00 2E 00 61 00 76 00 64 00 78 00 00 00	x.....a.v.d.x...
005959AA	2E 00 76 00 6D 00 63 00 78 00 00 00 2E 00 69 00	..v.m.c.x.....i.
005959BA	73 00 6F 00 00 00 52 00 65 00 73 00 74 00 61 00	s.o...R.e.s.t.a.

FIGURE 21. Memory view of the hardcoded secondary extension list.

Before encrypting, the worker compares the file's existing content against the marker string DLOGFILE0001 with memcmp(), proceeding with encryption only when the bytes do not match.

Section 3.2 noted that rstrtmgr.dll (the Windows Restart Manager) is loaded. It is used here: the routine that opens a target file for reading (0x00439430), named KillFileOwner, calls into Restart Manager when the initial CreateFileW() fails. Against a locked file, C:\DumpStack.log.tmp, the initial open failed with GetLastError = 32 (ERROR_SHARING_VIOLATION), and the file was skipped.

10.4 The Ransom Note: Content, Branding, and a Traceable Identifier

Before encrypting files in a directory, DragonForce writes a readme.txt there. The note opens by naming the operation directly: "We The DragonForce Ransomware Cartel offer you to make a deal with us." This self-identification corroborates the background in Section 1.1: external reporting describes DragonForce restructuring into a "ransomware cartel" affiliate model in March 2025.

The note also states a specific deadline and offers to "get a list of the files stolen from you" - an explicit double-extortion claim. No data-exfiltration mechanism, outbound command-and-control channel, or file-upload path appears in the observed execution path. The claim is not corroborated by this sample's demonstrated behavior.

Channel	Value
Negotiation portal (Tor)	3pktcrbcmssvrnwe5skburdwe2h3v6ibdnn5kbjqihsg6eu6s6b7ryqd.onion
Leak blog (Tor)	z3wqggtxft7id3ibr7srivv5gjof5fwg76slewnzwwakjuf3nlhukdid.onion
Tox support ID	1C054B722BCBF41A918EF3C485712742088F5C3E81B2FDD91ADEA6BA55F4A856D90A65E99D20

```

1 Good afternoon,
2
3 As you can see you have been attacked by a ransomware program! We The DragonForce Ransomware Cartel offer you to make a deal with us. We can make a deal with you, all you need to do is contact us by following the
4 instructions below.
5 We are in no way connected to politics, we always keep our word. You have a chance to decrypt your files and avoid being published on our blog! Use this opportunity and also don't waste your time.
6 The approximate date of deletion of the decryptor program, as well as publication on our blog 24/05/2026 00:00 UTC.
7
8 - # 1 Communication Process,
9
10 In order to contact us you need to click on the special link below, which is listed in #2.
11 After that the negotiation process begins, in which you have the opportunity to request several things from us,
12
13 1. make a test decrypt.
14 2. get a list of the files stolen from you.
15
16 At the conclusion of our negotiations we agree on a price, we set the price ourselves based on your income/your insurance.
17 We scrutinize your documents and are well aware of how much income your company has per year.
18
19 - # 2 Access to the meeting room,
20
21 To access us please download Tor Browser which is available here. (https://www.torproject.org/)
22 Once you download the special anonymous browser you need to follow this link, http://jpktrcbmssvrmetskburdwzthv6ibdn5k5qihsg6u6sb7ryqd.onion
23 Your unique ID: C09FC6C02B44E1C09FC6C02B44E1 - use it to enter our meeting room.
24
25 - # 3 Additional Support Contacts,
26
27 Tox: 1C05487228BF41A918EF3C485712742088F5C3E8182F0091ADEAGBA55F4A856D9A65E99D0E
28
29 - # 4 Recommendations,
30
31 Do not try to recover your files with third-party programs, you will only do harm.
32 Do not turn off / reboot your computer.
33 Be courteous in our meeting room.
34 Do not procrastinate.
35
36 - # 5 Blog and News,
37
38 Blog: http://2wmggtxtf7idibz7srivvgjefsfwg26slwzwaakjufnlnhukdid.onion
39 Dragonnews: http://2wmggtxtf7idibz7srivvgjefsfwg26slwzwaakjufnlnhukdid.onion/news

```

FIGURE 22. Generated readme.txt showing the DragonForce branding, contact channels.

10.5 File Encryption: Two Different Key Models

This is the most important cryptographic distinction in the whole document. Per-file key material is generated at runtime:

```

CryptGenRandom(hProv, 32 bytes) --> fresh 32-byte per-file key material
CryptGenRandom(hProv, 8 bytes) --> fresh 8-byte per-file state material
|
v
InitializeChaChaState() + AppendChaChaState() --> a NEW ChaCha20 state
                                                for this encryption operation

```

This newly generated key/state material is used with the same ChaCha20Crypt() core to encrypt the file's own content. Immediately after generation, DragonForce calls CryptEncrypt() (0x004335E0) with the RSA public key from Section 10.1 to protect the per-file material before appending it to the target file.

```

SetFilePointerEx(FILE_END)
|
v
WriteFile(0x20C / 524 bytes) --> RSA-wrapped per-file key/state material
|
v
SetFilePointerEx(...)
|
v
WriteFile(0x0D / 13 bytes) --> Encryption mode and file-size metadata footer
|
v
SetEndOfFile()

```

Memory inspection of the second write at 0x058BE26E reveals the exact structural layout of the 13-byte trailing footer:

```
058BE26E: 64 00 00 00 00 00 15 00 00 00 00 00 00
```

- **0x64:** Decimal 100, acting as the encryption percentage flag (indicating 100% full encryption in this execution path, in contrast to partial/chunked modes).
- **0x15:** 21 bytes in decimal, representing the original unencrypted file size of Victim.txt, enabling the decryptor to restore exact file bounds after stripping the trailer.

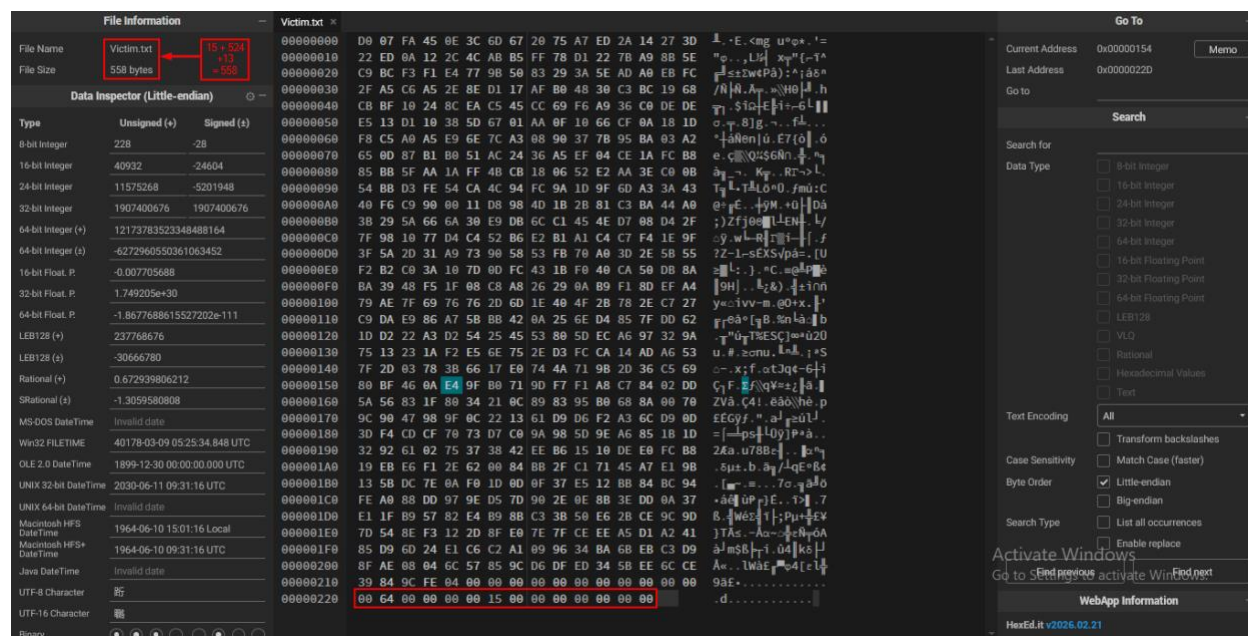


FIGURE 23. A full-file encryption event followed by the appended 524-byte RSA-wrapped per-file material and the final 13-byte footer write.

10.6 Renaming and Finalization

After encryption, DragonForce generates a new, randomized filename from a decrypted alphabet string and appends the configured .df_win extension:

```
Original: Victim.txt
New name: 5stq3an4ztrbkhnT.df_win
```

This is applied with MoveFileW(). The worker then moves to the next file, repeating Sections 10.3 - 10.6 until the directory tree is exhausted, then continues to the next task pulled from its queue.

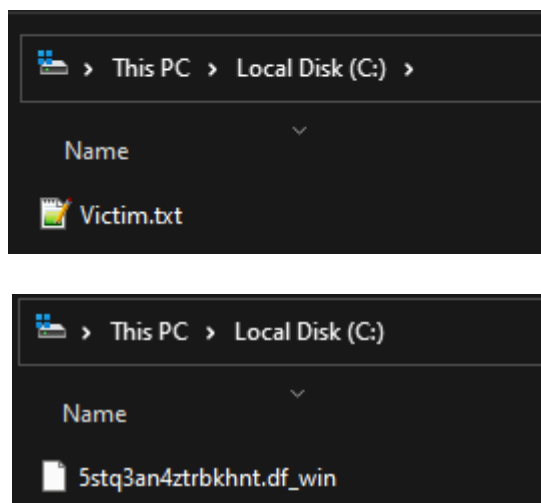


FIGURE 24. Before-and-after filesystem view. An original filename renamed to a randomized .df_win name after encryption.

Function	Address	Role
InitializeEncryptionWorkers()	0x00486B60	Creates the 16 encryption worker threads
EncryptionWorker()	0x004866F0	Worker entry point that sets up crypto context and consumes tasks
AcquireCryptoProviderContext()	0x00485BE0	Acquires the CryptoAPI provider context
VirtualizationExtensionCheck()	0x0042E510	Routes 200+ database/VM extensions into the 20% partial-encryption path
RSAProcessing()	0x004335E0	CryptGenRandom() + CryptEncrypt(), result appended by 0x00433E80
FileRename()	0x00444370	Generates the randomized name, appends .df_win, calls MoveFileW

11. Shutdown and Cleanup

After the encryption-worker stage completes, the main thread winds down the background workers before continuing into desktop customization (Section 12).

11.1 Terminating Idle Encryption Workers

Before waiting on the encryption-worker batches, the main thread actively shuts down workers that have nothing left to do - a second mechanism beyond the Sleep(0x1388) idle loop from Section 10.2. A

function at 0x004872E0 walks the drive-queue's worker handles; a worker that finds the queue-entry's +0x18 status field resolving to zero calls ExitThread() instead of proceeding into file processing.

```
mov eax, dword ptr ss:[esp+2C]
mov edx, dword ptr ds:[eax+18] ; a status/flag field
test edx, edx
jne sample.486A37 ; flag set -> different path
...
call ExitThread ; flag clear here -> worker exits
```

11.2 Confirming the Two Worker Batches

The main thread calls WaitForMultipleObjects() twice in succession, once for each batch of eight encryption-worker handles. With only two local drives discovered, the first batch finished quickly and sat idle. The second batch, working through more than eight discovered network shares, remained active substantially longer.

11.3 Stopping the Process-Monitoring Threads

A function at 0x0047A820 logs "Stopping proc," then calls WaitForSingleObject() on each of the two process-monitoring handles from Section 7. Once both have exited, it logs "Stopped proc" and closes the handles.

```
WaitForMultipleObjects() x2 --> both 8-worker encryption batches complete
|
v
"Stopping proc"
|
v
WaitForSingleObject() x2 --> the two Section 7 process-monitor threads exit
|
v
"Stopped proc" --> CloseHandle()
```

Function	Address	Role
EncryptionWorkerExecutionCompletion()	0x004872E0	Drives the shutdown of idle encryption workers and waits for execution completion.
MonitorWorkerExecutionCompletion()	0x0047A820	Waits for and closes the two Section 7 process-monitor threads

12. Desktop Customization

This stage begins only once the worker-thread cleanup in Section 11 has finished. `custom_icon` and `custom_wallpaper` (Section 4.3) drive a complete desktop-customization chain. The trace establishes the write attempts and the active-session wallpaper change.

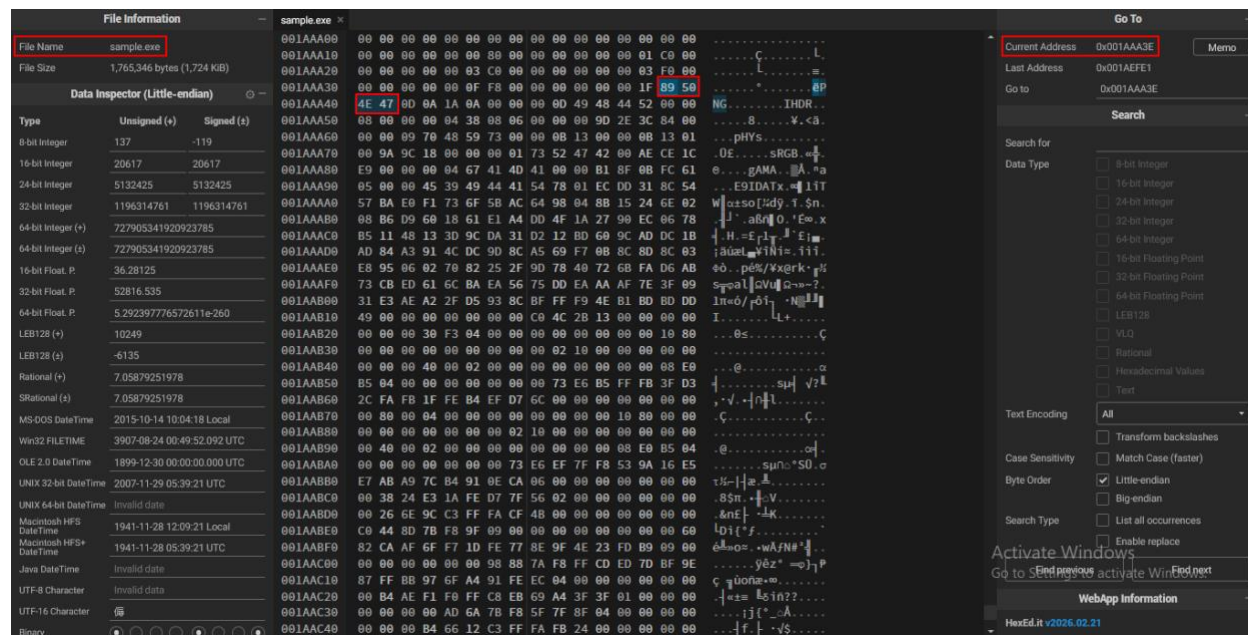
12.1 Extracting the Embedded Icon and Wallpaper

The routine at 0x0045A680 opens its own executable file with `_wfopen_s(path, "rb")` - not an external resource - seeks to a fixed offset, and reads a fixed-size block (0x423E bytes) with `fread()`.

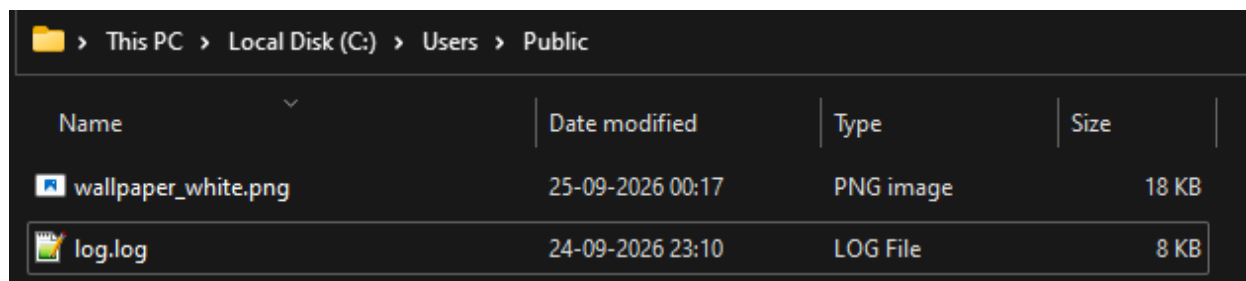
Both images are compiled directly into the binary's overlay. Static PE header analysis and runtime file I/O confirm that the sample's .tls section terminates at file offset 0x001A6800, where the 33.97 KB overlay begins:

1. **Icon extraction:** Begins at offset 0x001A6800 for a length of 0x423E (16,958 bytes), written to `C:\Users\Public\icon.ico`.
2. **Wallpaper extraction:** Begins immediately at offset 0x001AAA3E (0x001A6800 + 0x423E) for a length of 0x45A4 (17,828 bytes), written to `C:\Users\Public\wallpaper_white.png`. The bytes read begin with 89 50 4E 47 0D 0A 1A 0A, which is the PNG file signature.

The two extracted payloads sum to exactly 34,786 bytes (0x87E2), precisely accounting for the overlay region (0x001A6800-0x001AEFE2).



The screenshot displays the Immunity Debugger interface for `sample.exe`. The **File Information** pane on the left shows the file size as 1,765,346 bytes (1,724 KB). The **Data Inspector (Little-endian)** pane shows various data types, including 8-bit, 16-bit, 24-bit, 32-bit, and 64-bit integers, floats, and rationals. The **Hex Editor** pane shows the raw bytes of the file, with the address 0x001AAA3E highlighted. The **Go To** pane on the right shows the current address as 0x001AAA3E and the last address as 0x001AEFE1. The **Search** pane shows the search for "89 50 4E 47" (the PNG signature) and the results are displayed in the Hex Editor.



Name	Date modified	Type	Size
wallpaper_white.png	25-09-2026 00:17	PNG image	18 KB
log.log	24-09-2026 23:10	LOG File	8 KB

FIGURE 25. Hex view of the sample's own .exe file at the wallpaper read offset, showing the PNG signature alongside the extracted wallpaper_white.png.

12.2 Icon Association: A Registry Key, Not a Per-File Icon

Windows associates icons with file types/extensions rather than assigning them through the file-encryption routine itself. 0x004AF2E0 builds the subkey path .df_win\DefaultIcon and creates HKEY_CLASSES_ROOT\.df_win via RegCreateKeyExW(), then the nested DefaultIcon subkey, then sets its default value to C:\Users\Public\icon.ico.

Registry Action	Hive / Path	Value
RegCreateKeyExW	HKEY_CLASSES_ROOT\.df_win	(creates the key)
RegCreateKeyExW	HKEY_CLASSES_ROOT\.df_win\DefaultIcon	(creates the key)
RegSetValueExW	HKEY_CLASSES_ROOT\.df_win\DefaultIcon	(Default) = C:\Users\Public\icon.ico

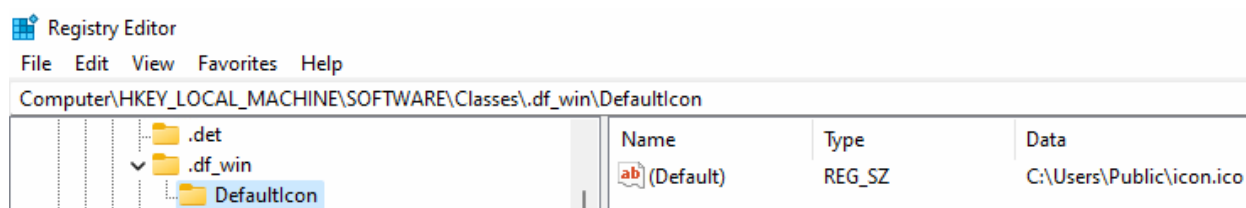


FIGURE 26. Registry Editor evidence showing creation of the .df_win\DefaultIcon key and its default value.

12.3 Wallpaper: Targets Enumerated User Contexts and the Active Session

The wallpaper mechanism (0x004A77C0) is more aggressive than the icon mechanism: rather than touching only the current user's settings, it enumerates the subkeys present under HKEY_USERS and attempts to set the wallpaper-related values for each enumerated profile/account context.

It opens HKEY_USERS with RegOpenKeyExW(), then walks every subkey with RegQueryInfoKeyW()/RegEnumKeyExW() - each subkey is a loaded user's SID (or a special account like

.DEFAULT). For every enumerated context, it logs "Found sid: %s" and calls two setters for Wallpaper and WallpaperStyle.

```
Found sid: .DEFAULT
Registry path: Control Panel\Desktop\DEFAULT
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
Found sid: S-1-5-19
Registry path: Control Panel\Desktop\S-1-5-19
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
Found sid: S-1-5-20
Registry path: Control Panel\Desktop\S-1-5-20
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
Found sid: S-1-5-21-2896843362-2627761479-3075785104-500
Registry path: Control Panel\Desktop\S-1-5-21-2896843362-2627761479-3075785104-500
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
Found sid: S-1-5-21-2896843362-2627761479-3075785104-500_Classes
Registry path: Control Panel\Desktop\S-1-5-21-2896843362-2627761479-3075785104-500_Classes
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
Found sid: S-1-5-18
Registry path: Control Panel\Desktop\ S-1-5-18
Set Wallpaper success = 0, 2
Set WallpaperStyle success = 0, 2
```

The "success = 0, 2" pairs are logged consistently for every enumerated SID, including accounts unlikely to have a normal desktop, such as .DEFAULT and S-1-5-18 (the Local System account). The 1st digit of the pair indicates the operation success and the 2nd indicates the LastError if the operation failed.

Error code 2 means "ERROR_FILE_NOT_FOUND".

After enumeration, the routine closes HKEY_USERS and calls SystemParametersInfoW(SPI_SETDESKWALLPAPER, ...) to apply the wallpaper to the current session immediately.

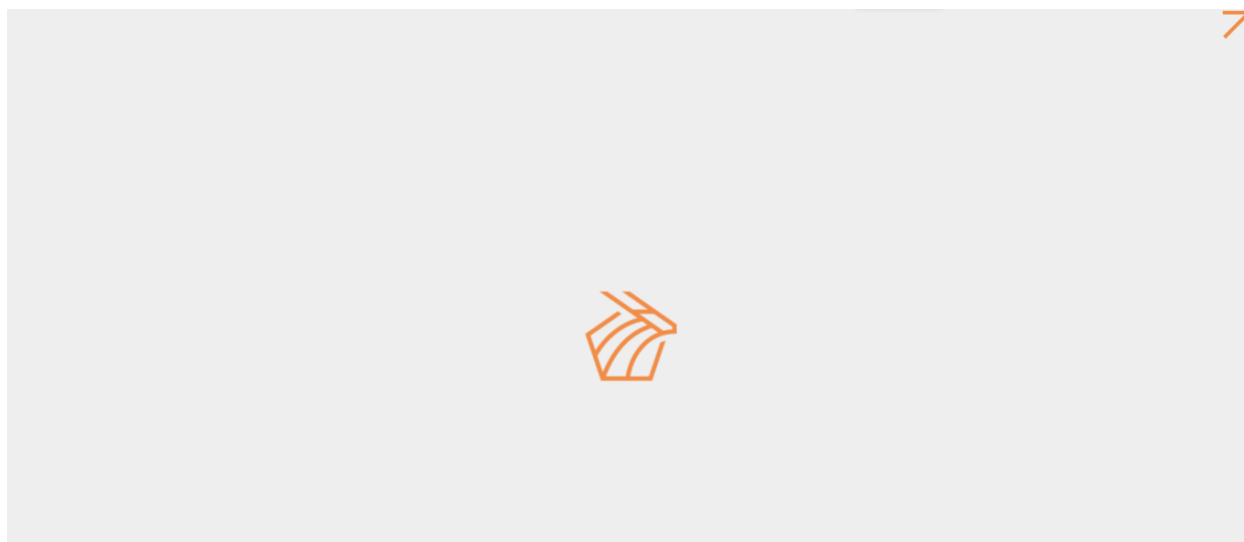


FIGURE 27. Active Interactive Desktop Following Execution of `SystemParametersInfoW(SPI_SETDESKWALLPAPER)`: Demonstrates successful visual defacement, replacing the user background with the extracted off-white canvas featuring DragonForce's central geometric orange dragon crest.

12.4 Program Completion

After the `WallpaperRegistration()` returns, the sample logs a final "Finish" entry, flushes and closes the log file through `0x004A56D0`, frees its remaining memory, and returns from `DragonForce_Main()`. No explicit `ExitProcess()` call was observed. Execution returns through the normal CRT shutdown path.

Function	Address	Role
<code>IconAndWallpaperExtraction()</code>	<code>0x0045A680</code>	Reads/writes both embedded images, drives icon and wallpaper setup
<code>IconRegistration()</code>	<code>0x004AF2E0</code>	Builds the <code>.df_win\DefaultIcon</code> path and drives key creation/value setting
<code>WallpaperRegistration()</code>	<code>0x004A77C0</code>	Enumerates <code>HKEY_USERS</code> , drives per-SID wallpaper application
<code>FinalLogFlush()</code>	<code>0x004A56D0</code> (via <code>0x004CD8C0</code>)	Writes and closes the log file at program end

13. Analytical Limitations and Unresolved Areas

This section defines the boundaries of what the analysis establishes.

13.1 Static Code Paths Never Dynamically Exercised

Item	Section	What's Confirmed vs. Not
DriverModeCheck() (0x0048AC30)	6.3	Contains decoded strings for truesight.sys and rentdrv2.sys (both publicly associated with BYOVD techniques), but the branch that would use them was never taken in this run. Static presence only, not demonstrated behavior.
CryptoAPI fallback provider strings	10.1	Alternate provider name strings and the ExitThread() failure path exist in the binary. The primary provider succeeded on first attempt, so neither was exercised.

13.2 Purposes That Remain Unresolved

Item	Section	Status
encrypt_mode: 10	4.3	Numeric meaning of this mode value not independently established.
Software\Microsoft\Manager registry key, sequence, next	4.4	Recovered strings. No confirmed read/write operation was observed against them.
EnumerateExplorerProcess() stored PID	6.4	PID located and stored via Toolhelp snapshot. No subsequent usage observed in desktop routine.
WSAIocctl() control code 0xC8000006	9.2	Exact named constant and malware-specific purpose not independently established.
Mersenne Twister-related function chain (0x004888B0)	4, Appx. A	Processes a string decoding to "mt19937." Relationship to key material or another subsystem not established.

13.3 Imported Capabilities Not Dynamically Exercised

Static analysis of the binary's PE import table shows 236 imported functions across 11 DLLs. Several imported capabilities never appeared in the runtime trace and are listed here as static-only observations rather than demonstrated behavior.

Function(s)	Category	Note
CreateServiceW, OpenSCManagerW, StartServiceW, ControlService, DeleteService, QueryServiceStatus	Service Control Manager	Full SCM access is imported. No service-related activity was observed anywhere in the dynamic trace.

Function(s)	Category	Note
CreateProcessWithTokenW, DuplicateTokenEx, InitializeSecurityDescriptor, SetSecurityDescriptorDacl	Token / security descriptor manipulation	Consistent with a SYSTEM-impersonation or privilege-escalation path. Never exercised in this run.
DeviceIoControl	Driver I/O	Static corroboration that DriverModeCheck() (Section 6.3) is a real mechanism, although the branch was not taken in this run.
Wow64DisableWow64FsRedirection, Wow64RevertWow64FsRedirection	WOW64 filesystem redirection	Would let this 32-bit process access the real System32 folder on the 64-bit host. Never observed being used.
CreateTimerQueue, CreateTimerQueueTimer, DeleteTimerQueue	Timer-based scheduling	A plausible candidate for the disabled schedule_job configuration value. That link was not independently established.
QueryPerformanceCounter	High-resolution timing	A plausible PRNG-seed input to the unresolved mt19937-related function chain. That link was not independently established.

13.4 Scope Limitations of This Analysis

Command-line behavior was only partially exercised. The binary contains handlers for -p, -m, -log, -size, and -nomutex, but the analyzed sample was executed without arguments. Section 3.4 documents each switch's function from published analysis of the shared Conti codebase. DragonForce's own implementation of that behavior was not dynamically demonstrated in this run.

No initial-access or delivery mechanism was analyzed. This document begins at process execution. How this sample would typically reach a target host is outside its scope.

No command-and-control channel was observed. Every network operation documented in Section 9 is local-network discovery and propagation. No outbound communication to an external server was seen in this trace.

Single-sample, limited-run analysis. Findings are based on one binary, analyzed dynamically across a small number of executions. Values described as fixed, such as the ChaCha20 key, are fixed across the observed runs rather than across every DragonForce build or deployment.

Program completion was observed, though not via an explicit `ExitProcess()` call. Section 12.4 traces the sample through a final "Finish" log entry, a last log-file flush and close, and a return out of `DragonForce_Main()`, consistent with a normal exit through the CRT shutdown path.

14. MITRE ATT&CK Mapping

This table maps only behavior established elsewhere in this report to corresponding MITRE ATT&CK techniques.

Technique ID	Technique Name	How DragonForce Does This	Section
T1027	Obfuscated Files or Information	Strings encoded with a reversible byte-wise transform	3.3
T1140	Deobfuscate/Decode Files or Information	ChaCha20-based decryption of the embedded configuration blob	4.2
T1082	System Information Discovery	<code>GetNativeSystemInfo()/IsWow64Process()</code> architecture checks	6.3, 8.1
T1033	System Owner/User Discovery	Token-based SID processing and <code>LookupAccountSidW()</code>	6.1
T1057	Process Discovery	<code>NtQuerySystemInformation()</code> -based enumeration against a priority list	7.2
T1562.001	Impair Defenses: Disable or Modify Tools	Attempts to terminate configured priority-list processes, including security software	7.2
T1490	Inhibit System Recovery	Queries <code>Win32_ShadowCopy</code> and constructs a <code>WMIC.exe</code> delete command	8.2
T1016	System Network Configuration Discovery	ARP-cache (<code>GetIpNetTable</code>) based discovery of internal-network hosts	9.2
T1135	Network Share Discovery	<code>NetShareEnum()</code> -based enumeration of SMB shares on discovered hosts	9.3
T1021.002	Remote Services: SMB/Windows Admin Shares	Reaches and processes files through reachable SMB shares, including <code>C\$</code>	9.3
T1486	Data Encrypted for Impact	Per-file ChaCha20 encryption with RSA-wrapped runtime-generated key material, plus randomized renaming	10.5 - 10.6

Technique ID	Technique Name	How DragonForce Does This	Section
T1112	Modify Registry	Creates the .df_win\DefaultIcon association and attempts wallpaper writes for enumerated HKEY_USERS contexts	12.2 - 12.3
T1491.001	Defacement: Internal Defacement	Applies a custom wallpaper to the active session with SystemParametersInfoW()	12.3
T1012	Query Registry	Enumerates HKEY_USERS subkeys for subsequent wallpaper-setting operations	12.3

15. Indicators of Compromise

File Paths and Names

Indicator	Type	Section
C:\Users\Public\log.log	Encrypted runtime log	5.1
C:\Users\Public\icon.ico	Custom icon extracted from the sample's own executable	4.4, 12.1
C:\Users\Public\wallpaper_white.png	Custom wallpaper extracted as a PNG	4.4, 12.1
readme.txt	Ransom note written in processed directories	10.4
.df_win	Extension appended by the file-renaming path	4.3, 10.6
DLOGFILE0001	File-content marker checked before encryption	10.3, 13.2

Registry

Indicator	Notes	Section
Software\Microsoft\Manager	Recovered from the decrypted configuration blob.	4.4, 13.2
HKEY_CLASSES_ROOT\.df_win\DefaultIcon	Set to icon.ico - associates the .df_win file type with the extracted icon	12.2

Indicator	Notes	Section
Wallpaper / WallpaperStyle under enumerated HKEY_USERS contexts	Attempted for every enumerated SID.	12.3

Process and Synchronization Artifacts

Indicator	Notes	Section
hsfjuukjzloqu28oajh727190	Mutex name passed to CreateMutex() during the analyzed execution	7.3
MsMpEng.exe, Firefox.exe, and 36 other priority-list processes	Full list in Section 4.3	4.3, 7.2

Configuration Values

Value	Observed Data	Section
build_key	9E3AA5CEC257EA66	4.3, 10.4
instance_key	561C24DEEECC136E	4.3, 10.4
Full-encrypt threshold	2,097,152 bytes (2 MB)	4.3
Secondary-extension partial-encryption rate	20%	4.3, 10.3

Network Behavior

Behavior	Detection Angle	Section
SELECT * FROM Win32_ShadowCopy followed by a WMIC.exe shadowcopy delete command	Process-creation telemetry for wmic.exe would be high-value if executed	8.2
ARP-cache read followed by SMB share enumeration against multiple internal hosts	Useful for identifying local-network discovery and target publication	9.2 - 9.3

16. Quick Heal Detections

Seqrite/Quick Heal detects the analyzed sample through its multi-module detection stack, leveraging multiple detection mechanisms to provide layered protection against the identified threat.

Appendix A. Function and Address Reference

Every function named in Sections 1 - 12 is listed here with its address and a one-line role.

Function	Address	Role	Section
TLS Callback 1 / 2	0x004D96E0 / 0x004D9730	Standard runtime init, no DragonForce-specific behavior	3.1
PE Entry Point	0x004014B0	Hands off to CRT startup	3.1
CRT Startup	0x0058E3D0	Compiler-generated init that retrieves the command line	3.1
DragonForce_Main()	0x0045E160	Start of malware-specific execution	3.1
InitializeApiResolver()	0x00407D10	Normalizes key strings, resolves LoadLibraryA	3.2
LoadRequiredModules()	0x00401AB0	Decrypts and loads the 11 required DLLs	3.2
ResolveModuleExports()	0x004016E0	Calls GetProcAddress, validates against .idata import thunks	3.2
CompareCommandLineArgument()	0x0048940	Compares a decoded switch against the supplied argument (address retained as observed)	3.4
PrepareConfigurationData()	0x00488A70	Builds the ChaCha20 state, then decrypts and parses the configuration	4
DecryptEmbeddedConfiguration()	0x00487FE0	Decrypts the embedded configuration blob	4.2
ChaCha20Crypt()	0x00487990	Shared ChaCha20 core, used for config decrypt AND log encrypt	4.2, 5.2

Function	Address	Role	Section
mt19937-related function chain	0x004888B0	Processes a string decoding to "mt19937," purpose not established	4, 13.2
InitializeLogFilePath()	0x00489F80	Decrypts and allocates the log.log path string	5.1
InitializeLogFile()	0x00494070	CreateFileW + SetFilePointer on log.log	5.1
PrepareLogEncryptionState()	0x00494110	Writes build_key/instance_key header, then builds log ChaCha20 state	5.1
WriteEncryptedLogEntry()	0x00495440	Flag-gated wrapper into PrepareEncryptedLogEntry	5.2
PrepareEncryptedLogEntry()	0x00494270	Builds timestamp+thread prefix, assembles full message	5.2
EncryptLogBuffer()	0x00487FB0	Encryption wrapper forwarding into ChaCha20Crypt()	5.2
LogFormattedMessage()	0x004A56A0	Flag-gated wrapper into PrepareEncryptedLogEntry	5.2, 6.1
CollectUserAndHostInformation()	0x0048CAA0	Recovers account/host-related strings used by the logging path	6.1
CheckProcessElevation()	0x00492310	Confirms process token is elevated	6.2
FindLastPathSeparator()	0x004F6110	Splits executable path into directory/filename	6.3
DriverModeCheck()	0x0048AC30	Reads a mode byte and operates on driver-related strings. Branch not met in this run	6.3
EnumerateExplorerProcess()	0x0046D980	Locates and stores explorer.exe PID	6.4
InitializeProcessMonitoring()	0x00479B30	Resolves NtQuerySystemInformation, creates the two worker threads	7.1

Function	Address	Role	Section
PriorityProcessWorker()	0x0047DE00	Walks priority list, terminates matches, sleeps 250ms, repeats	7.2
SecondaryPriorityProcessWorker()	0x0047E8A0	Same static logic. Only reached its sleep loop in the observed run	7.2
EnumerateAndMatchProcess()	0x0047DDB0	Compares a target process name against the live process list	7.2
TerminateTargetProcess()	0x00475EC0	Opens and terminates a matched process	7.2
ManageShadowCopies()	0x00409280	Drives the COM/WMI shadow-copy query and deletion-command construction	8.1 - 8.2
EnumerateLogicalDrives()	0x0044CEE0	Enumerates local drives, queues each via PutTask()	9.1
PutTask()	0x00486BF0	Shared task-queue insertion, used for both drives and shares	9.1, 9.3
EnumerateInternalIPs()	0x00463B80	Walks the ARP cache, filters to private/link-local ranges	9.2
EnumerateAndAppendShares()	0x00466760	Runs NetShareEnum(), builds selected UNC paths and calls PutTask()	9.3
InitializeEncryptionWorkers()	0x00486B60	Creates the 16 encryption worker threads	10.1
EncryptionWorker()	0x004866F0	Worker entry point that sets up crypto context and consumes tasks	10.1 - 10.2
AcquireCryptoProviderContext()	0x00485BE0	Acquires the CryptoAPI provider context	10.1
RSAProcessing()	0x004335E0	Calls CryptEncrypt() on a buffer containing generated per-file material	10.5
EofDataWrite()	0x00433E80	Writes the 0x20C-byte RSA record and the 0x0D-byte footer, then SetEndOfFile()	10.5

Function	Address	Role	Section
VirtualizationExtensionCheck()	0x0042E510	Routes 200+ extensions into the 20% partial-encryption path	10.3
KillFileOwner() (Restart Manager caller)	0x00439430	Opens the target file, invokes Restart Manager on failure	10.3, 10.5
FileRename()	0x00444370	Generates the randomized name, appends .df_win, calls MoveFileW	10.6
IconAndWallpaperExtraction()	0x0045A680	Reads and writes both embedded images, drives icon/wallpaper setup	12.1
IconRegistration()	0x004AF2E0	Builds the .df_win\DefaultIcon path and drives key creation/value setting	12.2
WallpaperRegistration()	0x004A77C0	Enumerates HKEY_USERS, drives per-SID wallpaper application	12.3
FinalLogFlush()	0x004A56D0 (via 0x004CD8C0)	Writes and closes the log file at program end	12.4
EncryptionWorkerExecutionCompletion()	0x004872E0	Drives the shutdown of idle encryption workers and waits for execution completion.	11.1
MonitorWorkerExecutionCompletion()	0x0047A820	Waits for and closes the two Section 7 process-monitor threads	11.3

Appendix B. Selected Platform and API References

This appendix summarizes the documented Windows behavior of the APIs and structures cited throughout the report.

API / Structure	Documented Behavior	Cited In
IsWow64Process	Determines whether a 32-bit process is running under WOW64 on a 64-bit host	6.3
TOKEN_ELEVATION / TokenElevation	A GetTokenInformation() class whose TokenIsElevated field indicates elevation	6.2

API / Structure	Documented Behavior	Cited In
WaitForSingleObject	A timeout of 0 tests the object's state and returns immediately rather than block	7.3
CreateMutex / CreateMutexA	If bInitialOwner is TRUE and the thread creates the mutex, it receives initial ownership	7.3
CreateThread	lpStartAddress specifies where execution begins, dwCreationFlags 0 is runnable	7.1, 10.1
IWbemLocator / CoCreateInstance(CLSID_WbemLocator)	The standard entry point for a WMI client	8.1
Win32_ShadowCopy	A WMI class representing Volume Shadow Copies, queryable via WQL	8.2
GetNativeSystemInfo	Retrieves processor architecture (9 = PROCESSOR_ARCHITECTURE_AMD64)	8.1
GetLogicalDriveStringsW	Fills a buffer with one string per valid logical drive	9.1
GetIpNetTable	Retrieves the IPv4-to-physical-address mapping table (the ARP cache)	9.2
NetShareEnum	Enumerates shared resources on a specified server	9.3
CryptAcquireContextA	Obtains a handle to a cryptographic service provider context	10.1
CryptImportKey	PUBLICKEYBLOB (RSA1 tag) is the documented format for a plaintext public key	10.1
CryptGenRandom	Generates cryptographically random bytes using the CSP's RNG	10.5
CryptEncrypt	Commonly used to wrap a symmetric session key with an asymmetric public key	10.5
Restart Manager (RmStartSession et al.)	Identifies and shuts down applications holding a file open	3.2, 10.3
RegCreateKeyExW / RegSetValueExW	Standard Windows Registry APIs for creating and writing key values	12.2, 12.3
SystemParametersInfoW (SPI_SETDESKWALLPAPER)	Changes the desktop wallpaper. SPIF_UPDATEINIFILE persists the change.	12.3



Quick Heal Technologies Limited

Phone: 1800-212-7377 | info@seqrite.com | www.seqrite.com |